minGPT: Adım Adım, FPGA Potansiyeli ve LittleBit · konuşma metni

Nasıl kullanılır: Her başlık bir slayta karşılık gelir; numaralar slayt numarasıdır. Aynı metin, sunumda her slaytın konuşmacı notunda da var. Süre kısaysa her bölümün ilk iki-üç cümlesini söyleyip slayttaki şekli göstermek yeterli. Metnin sonunda hocanın sorabileceği sorular ve kısa cevapları var.

Akış: 1–2 giriş · 3–5 temel kavramlar · 6–9 minGPT nedir · 10–29 minGPT adım adım (10–18 embedding ve matematiği, 20–26 attention) · 30–32 çıkış, unembedding ve KV cache · 33–36 FPGA potansiyeli · 37–54 araştırma konusu: LittleBit (38 terimler, 40 quantization, 44 LoRA ile karşılaştırma) · 55 özet.

---

1. minGPT: adım adım

Hocam merhaba. Bitirme projemin ilk bölümünü anlatacağım. Projenin bütününde Karpathy'nin minGPT'sini ve aynı soydan gelen modelleri bir FPGA üzerinde çalıştırıyoruz. Ama bunu yapabilmek için önce modelin içinde tam olarak ne olduğunu anlamamız gerekiyordu. Bu sunum o yüzden yalnızca minGPT'ye ayrıldı. Bir language model'in ne yaptığından başlayacağım, minGPT'nin bir token'ı nasıl işlediğini her adımda formülüyle, şekliyle ve onu yapan kod satırıyla göstereceğim. Sonunda da bu yapının FPGA açısından nerede güçlü, nerede zorlayıcı olduğunu tartışacağım. Kendi donanım tasarımımızın ayrıntılarına bu sunumda girmiyorum; onu ayrı bir bölümde anlatacağım. Bir not: teknik terimleri bilerek İngilizce bıraktım, çünkü literatürde ve kodda bu isimlerle geçiyorlar; Türkçe karşılıklarını kullanırsam kodla eşleştirmek zorlaşıyor.

2. Bu sunum minGPT'yi sıfırdan kuruyor, FPGA açısından değerlendiriyor ve bir araştırma konusuyla bitiyor

Sunumun akışı altı adım. Önce temel kavramları kuracağım: language model ne demek, token ve embedding ne, bir modelin iyi olduğunu nasıl ölçüyoruz. İkinci olarak minGPT'nin ne olduğunu konumlandıracağım: yeni bir mimari değil, GPT-2'nin sade bir yeniden yazımı; modern LLM'lerden farkını ve 300 satırlık bir kodun nasıl olup da çalışan bir GPT olabildiğini anlatacağım. Üçüncü ve en uzun kısım minGPT'nin bir token'ı adım adım nasıl işlediği. Burada hocamın isteği üzerine embedding'in matematiğine ayrıca girdim: one-hot çarpımının neden bir satır seçimine indiği, gradient'in nasıl aktığı, pozisyon bilgisinin üç yolu ve bizim modelimizin öğrendiği gerçek embedding'ler. Attention'a da özellikle zaman ayırdım: önce hangi problemi çözdüğü, sonra fikri, formülü, elle hesaplanmış bir örnek ve

gerçek modelin ne yaptığı. Dördüncü olarak modelin metni nasıl ürettiğini ve KV cache fikrini anlatacağım. Beşinci olarak bu yapının FPGA açısından güçlü ve zayıf yanlarını tartışacağım. Son olarak araştırma konusu olarak verilen LittleBit makalesini, matematiğiyle birlikte, adım adım açacağım ve bunun bizim projemiz için ne anlama geldiğini tartışacağım.

3. Language model, geçmişe bakarak bir sonraki token'ın olasılık dağılımını tahmin eder

Önce en temel soru: language model ne yapar? Bir metnin, yani bir token dizisinin olasılığını verir. Bunu doğrudan hesaplamak imkânsız gibi görünür, ama olasılık kuramındaki zincir kuralı işi basitleştiriyor. Bir dizinin olasılığı, her adımda "şimdiye kadar görülenlere göre bir sonraki token'ın olasılığı"nın çarpımına eşittir. Burada önemli olan şu: bu bir yaklaşıklık değil, tam bir eşitlik. Dolayısıyla bir language model'in öğrenmesi gereken tek şey bir fonksiyon: geçmişi al, bir sonraki token için bir olasılık dağılımı ver. ChatGPT'den minGPT'ye kadar bütün GPT tarzı modeller tam olarak bunu yapar. Sağdaki örnek isim üreten bir modelden. Model "e, m, m" harflerini görmüş; şimdi dördüncü harf için 27 sembolün her birine bir olasılık veriyor. Değerler temsilî, ama mantık gerçek: "a" harfine yüksek olasılık verir, çünkü "emma" sık görülen bir isim. Metin üretmek de buradan çıkar: bu dağılımdan bir harf seçeriz, dizinin sonuna ekleriz ve aynı işlemi tekrarlarız. Her yeni token bir öncekinin üzerine kurulduğu için buna autoregressive generation denir. Bu sunumun geri kalanında hep bu tek işlemin, yani "geçmişten sonraki token dağılımını hesaplama"nın içini açacağım. Donanım açısından da işin özü bu: bir çağrıda bir token girer, sonraki token için bir dağılım çıkar.

4. Token bir sembolün sayı kimliğidir; embedding bu kimliği öğrenilmiş bir vektöre çevirir

Model metni doğrudan göremez; önce metni token'lara bölmemiz gerekir. Token, bir sembolün sayı kimliğidir. İsim verisinde her harf bir token: nokta 0, a 1, b 2 diye gider, e 5, m 13. Ama bu sayılar yalnızca etiket. m'nin 13 olması, e'nin iki buçuk katı olduğu anlamına gelmez. Bu sayıları doğrudan bir ağa verirsek ağ olmayan bir büyüklük ilişkisi öğrenmeye çalışır. Çözüm embedding: her token için C tane sayıdan oluşan bir vektör tutarız. Bütün bu vektörler V satır, C sütunluk bir tabloda durur. Token id'si tablonun hangi satırını okuyacağımızı söyler. Bu vektörlerin değerleri de diğer weight'ler gibi training sırasında öğrenilir. Benzer bağlamlarda geçen token'ların vektörleri birbirine yaklaşır; örneğin sesli harflerin vektörleri birbirine benzemeye başlayabilir. Yani id'nin taşıyamadığı "benzerlik" bilgisini vektör taşır. Matematiksel olarak embedding, token'ın one-hot vektörüyle tablonun çarpımıdır. One-hot vektörde yalnızca bir tane 1 olduğu için bu çarpım tablonun o satırını seçmekten ibarettir. Bu yüzden pratikte embedding bir hesap değil, bir tablo okuması. minGPT kodunda bunu nn.Embedding yapar: içine token id'leri verirsiniz, her biri için C

boyutlu vektörü döner. Bu ayrıntı FPGA açısından önemli; tablo okuması, donanımda bir bellekten tek adres okumak demek. Hiç çarpma gerekmez.

5. Model, doğru token'a verdiği olasılıkla ölçülür (cross-entropy loss) ve bu loss'u küçültecek şekilde eğitilir

Bir modelin iyi olup olmadığını nasıl ölçeriz? Her pozisyonda modelin doğru token'a verdiği olasılığa bakarız. Model doğru cevaba yüksek olasılık veriyorsa iyidir. Bu olasılığın eksi doğal logaritmasını alıp bütün pozisyonlarda ortalarız; buna cross-entropy loss denir. Logaritma doğal olduğu için birimi nat/token. Tablodaki satırlar sezgi veriyor. Hiçbir şey bilmeyen bir model 27 harfe eşit olasılık verir; doğru harfe 1/27 düşer, loss ln 27, yani 3,30 olur. Model iki aday arasında kararsızsa, yani doğruya 0,5 veriyorsa, loss 0,69. Doğruya 0,9 veriyorsa 0,11. Mükemmel bir model sıfır loss alır. Logaritma küçük olasılıkları çok sert cezalandırır: model emin olup yanılırsa loss çok büyür. Training de bu sayıyı küçültmek demek. Sağdaki dört satır minGPT'nin training döngüsünün kalbi. Birinci satırda model tahminini yapar ve loss'u hesaplar. İkinci satırda eski gradient'ler sıfırlanır. Üçüncü satırda backward çağrısıyla PyTorch, loss'un her weight'e göre türevini, yani gradient'ini otomatik hesaplar. Dördüncü satırda optimizer, burada AdamW, her weight'i loss'u azaltacak yönde küçük bir adım kaydırır. Bu döngü binlerce kez tekrarlanır. Bir ayrımı baştan netleştirmek istiyorum: training tamamen bilgisayarda, floating point sayılarla yapılıyor. FPGA tarafında hiçbir zaman training yapmıyoruz; yalnızca eğitilmiş modeli çalıştırıyoruz. Buna inference denir. Bu ayrım önemli, çünkü inference'ın hesabı training'inkinden çok daha basit: ne gradient ne optimizer var, sadece ileri yönde hesap.

6. minGPT'nin tamamı üç parça: embedding, L tane aynı block ve bir output katmanı

Şimdi minGPT'nin kendisine geçiyorum. Bütün model aslında bu tek satırlık şemadan ibaret. Soldan sağa okuyalım. Önce embedding: token'ın vektörü ile pozisyonunun vektörü toplanır. Sonra L tane block gelir. Bütün block'lar aynı yapıda, sadece weight'leri farklı. Her block iki alt adım yapar. Birincisi attention: her token, kendisinden önceki token'lardan bilgi toplar. Bu modelin token'lar arasında bilgi taşıyan tek yeri. İkincisi MLP: her token kendi vektörünü kendi içinde işler, diğer token'lara bakmaz. İki alt adımın da önünde bir LayerNorm var ve ikisinin de sonucu x'in yerine geçmez, x'in üstüne eklenir; buna residual connection denir. Block'lardan sonra son bir LayerNorm ve lm_head adlı bir katman gelir. lm_head, vocabulary'deki her token için bir skor üretir; bunlara logit diyoruz. Logit'ler softmax'tan geçince bir sonraki token'ın olasılık dağılımı olur. Soldaki kod GPT.forward fonksiyonunun kendisi. Şemadaki her kutunun bir satıra karşılık geldiğini görüyorsunuz: wte token embedding, wpe position embedding, for döngüsü block'lar, ln_f son norm, lm_head çıkış. Bu sunum boyunca kullanacağım model minGPT'nin en küçük hazır

yapılandırması olan gpt-nano: 3 block, her block'ta 3 attention head, vektör boyutu 48, head başına 16 boyut. Vocabulary sadece 3 token, çünkü görevimiz 0, 1, 2 rakamlarını sıralamak; context en fazla 11 token. Toplam yaklaşık 85,7 bin parametre. Şimdi bu şemadaki her kutuyu sırayla açacağım; ama önce minGPT'nin ne olduğunu ve neden bu kadar kısa olduğunu konumlandırayım.

7. minGPT yeni bir mimari değil: GPT-2'nin eğitim amaçlı, sade bir yeniden yazımı

minGPT'nin ne olduğunu doğru konumlandırmak gerekiyor, çünkü "minGPT diğer LLM'lerden ne farklı yapıyor" sorusunun cevabı biraz şaşırtıcı: mimari olarak hiçbir şeyi farklı yapmıyor. Andrej Karpathy minGPT'yi 2020'de, GPT'nin nasıl çalıştığını göstermek için yazdı. Mimari, OpenAI'nin 2019'daki GPT-2'siyle birebir aynı. Bunun en güçlü kanıtı şu: minGPT'de from_pretrained diye bir fonksiyon var ve OpenAI'nin yayımladığı GPT-2 weight'lerini olduğu gibi yükleyip çalıştırabiliyor. Mimari bir parametre bile farklı olsaydı bu mümkün olmazdı. Peki modern LLM'ler, örneğin Meta'nın Llama 2'si ya da Karpathy'nin daha yeni nanochat'i, neyi farklı yapıyor? Tablo bunu özetliyor. Normalization'da LayerNorm yerine RMSNorm kullanıyorlar; ortalamayı çıkarmadan yalnızca büyüklüğe göre ölçekliyor, biraz daha ucuz. Pozisyon bilgisini ayrı bir tablodan eklemek yerine RoPE ile veriyorlar; q ve k vektörlerini pozisyona bağlı bir açıyla döndürüyor. MLP'de GELU yerine SwiGLU ya da ReLU kare kullanıyorlar. Attention'da GQA kullanabiliyorlar: birkaç query head'i aynı key ve value'yu paylaşıyor, bu da bellek tasarrufu sağlıyor. Bias'ları kaldırıyorlar. Generation sırasında KV cache kullanıyorlar; minGPT'nin generate fonksiyonu bunu yapmıyor, her adımda bütün diziyi baştan hesaplıyor. Ve tabii ölçek: gpt-nano 85 bin parametre, modern modeller milyarlarca. Ama asıl mesaj alttaki kutuda: çekirdek mekanizma aynı. Hepsi decoder-only transformer; embedding, tekrarlanan block'lar, her block'ta attention ve MLP, residual connection. Modern LLM'ler bu çekirdeğin ince ayarlı ve çok büyütülmüş hâli. Bu yüzden minGPT'yi baştan sona anlamak, hepsinin temelini anlamak demek. Bizim için seçim sebebi de bu: küçük, okunabilir ve tamamen anlaşılabilir.

8. 310 satır neden yetiyor? Ağır işi PyTorch yapıyor, transformer da kendini tekrar ediyor

Akla gelen doğal soru şu: milyar dolarlık modellerin temeli olan bir mimari nasıl oluyor da yorumlarıyla birlikte 310 satırlık bir dosyaya sığıyor? Bunun beş sebebi var ve beşi de koddan okunabiliyor. Birincisi: işin ağır kısmını PyTorch yapıyor. En zor kısım olan backpropagation, yani her weight için gradient hesabı, PyTorch'un autograd özelliğiyle otomatik. minGPT'de bu tek satır: loss.backward(). Linear katman, LayerNorm, softmax, AdamW optimizer, GPU kernel'leri; hepsi hazır. minGPT yalnızca bunların nasıl dizileceğini tanımlıyor. İkincisi: transformer kendini tekrar ediyor. Tek bir Block sınıfı yazılıyor ve

ModuleList ile L kez kopyalanıyor. 3 block da olsa 96 block da olsa kod aynı. Üçüncüsü: query, key ve value için üç ayrı katman yazmak yerine tek bir Linear katman üçünü birden hesaplıyor; çıkış üçe bölünüyor. Dördüncüsü bence en şık hilesi: birden çok attention head için döngü yazılmıyor. Tensor'ün boyutları view ve transpose ile yeniden düzenleniyor; head boyutu batch boyutu gibi davranıyor ve bütün head'ler tek bir matris çarpımıyla hesaplanıyor. Beşincisi training'in verimliliğiyle ilgili: causal mask sayesinde bir dizinin her pozisyonu yalnızca kendinden öncekilere bakıyor. Bu yüzden T uzunluğundaki bir dizide T tane "sonraki token" tahmini tek bir forward geçişinde, paralel olarak yapılabiliyor. Sağdaki kutu da madalyonun öbür yüzü: minGPT'de bilerek olmayanlar. KV cache yok, FlashAttention gibi hızlı kernel'ler yok, mixed precision yok, birden çok GPU ile training yok. Hepsi kodu uzatırdı. Amaç hız değil, okunabilirlik. Bu bizim için iyi bir başlangıç noktası, ama FPGA'ye taşırken bu eksiklerin bazılarını, özellikle KV cache'i, kendimiz eklememiz gerekecek.

9. Çalışan örneğimiz: gpt-nano altı rakamı küçükten büyüğe sıralamayı öğreniyor

Bu sunumdaki bütün örnekleri tek bir görev üzerinden vereceğim. Bu görev minGPT'nin kendi demo görevi: 0, 1 ve 2 rakamlarından oluşan altı rakamlık bir diziyi küçükten büyüğe sıralamak. Mesela girdi 2 0 1 1 0 2 ise beklenen çıktı 0 0 1 1 2 2. Burada ilginç olan, modelin sıralama algoritması bilmemesi. Model yalnızca bir language model; yani girdiyi ve çıktıyı yan yana koyduğumuz 12 token'lık diziyi görüyor ve her pozisyonda bir sonraki token'ı tahmin etmeyi öğreniyor. Soldaki kod eğitim örneğinin nasıl kurulduğunu gösteriyor. Girdi sıralanıyor, ikisi uç uca ekleniyor, 12 token oluyor. Modele ilk 11 token veriliyor; hedef ise bir kaydırılmış dizi, yani her pozisyonda bir sonraki token. Son satır önemli: girdi kısmındaki hedefler eksi bir yapılıyor. Bu, "burada loss hesaplama" demek. Çünkü girdinin bir sonraki rakamını tahmin etmek anlamsız, girdi rastgele. Model yalnızca çıktı kısmında, yani sıralı rakamları yazarken değerlendiriliyor. Bu görevi seçmemizin büyük bir avantajı var: olası girdi sayısı 3 üzeri 6, yani yalnızca 729. Örneklem almak yerine modeli bütün girdi uzayında test edebiliyoruz. Eğitilmiş gpt-nano 729 girdinin hepsini doğru sıralıyor. Şimdi bu modelin bir token'ı nasıl işlediğine adım adım bakalım.

10. Adım 1 · Token embedding ile position embedding toplanır: vektör hem "ne" hem "nerede" bilgisini taşır

Birinci adım embedding. Her pozisyon için iki vektörü topluyoruz. Birincisi token embedding: o pozisyondaki token'ın vektörü, yani "bu token ne?" sorusunun cevabı. İkincisi position embedding: pozisyonun vektörü, yani "bu token dizinin neresinde?" sorusunun cevabı. İkisi de öğrenilen tablolardan okunuyor; token embedding tablosunda vocabulary boyutu kadar, position embedding tablosunda en uzun context kadar satır var. İki vektör

eleman eleman toplanıyor ve sonuç block'lara giriyor. Şekildeki renkler sayıları temsil ediyor; amaç, iki bilginin tek bir vektörde birleştiğini göstermek. Peki position neden gerekli? Biraz sonra göreceğimiz attention, geçmişteki vektörlerin ağırlıklı bir toplamı. Toplama işlemi sıra bilmez: a artı b, b artı a ile aynı. Pozisyon bilgisi eklenmeseydi model "2 0 1" ile "1 0 2" dizilerini birbirinden ayırt edemezdi; sıralama gibi sırayla ilgili bir görevi hiç öğrenemezdi. Position embedding sayesinde aynı "0" token'ı birinci pozisyonda ve beşinci pozisyonda farklı vektörlerle temsil ediliyor. Kodda bu dört satır: arange ile 0'dan t'ye pozisyon indeksleri üretiliyor, iki tablodan vektörler okunuyor ve toplanıyor. Toplamın etrafındaki drop, dropout; training sırasında rastgele bazı elemanları sıfırlayarak ezberlemeyi önlüyor. Inference'ta ise hiçbir şey yapmıyor, yani modeli çalıştırırken bu satırı yok sayabiliriz. Bir not: modern modeller pozisyonu ayrı bir tabloyla değil, RoPE ile veriyor; minGPT'nin yolu daha basit, bir tablo okuması.

11. Embedding matrisi: vocabulary'deki her token'ın, training ile öğrenilen bir satırı vardır

Embedding'i en temel hâliyle tanımlayalım: embedding, ayrık bir sembolü, yani bir token ID'sini, sürekli bir vektör uzayındaki bir noktaya dönüştüren, öğrenilebilir bir fonksiyon. Neden gerekli? Model "kedi", "araba", "the" gibi token'ları doğrudan işleyemez; Transformer katmanlarının çalışabilmesi için bunların reel sayılardan oluşan vektörlere çevrilmesi gerekir. Bunu yapan şey embedding matrisi E. Vocabulary'de V tane token var ve vektör boyutu d olsun; E, V satır d sütunluk bir matris. Her satır bir token'ın vektörü. Örneğin V = 50 000 ve d = 4096 olan bir modelde E'de yaklaşık 205 milyon sayı var. Token ID'si x olan bir token için yaptığımız tek şey E'nin x'inci satırını almak; 1532 numaralı token için E[1532], 4096 reel sayıdan oluşan bir vektör. Bu sayıların tek tek insan açısından doğrudan bir anlamı yok; anlam, vektörlerin birbirine göre nerede durduğunda. Bir cümleyi düşünelim: "Murat yapay zekayı seviyor". Tokenizer bunu, örneğin, dört ID'ye çevirir; ID'ler temsilî. Her ID için E'den bir satır alıp alt alta koyarsak 4 çarpı 4096 boyutlu bir X matrisi elde ederiz. Birden çok cümleyi aynı anda işliyorsak, yani bir batch varsa, X üç boyutlu bir tensör olur: batch boyutu çarpı dizi uzunluğu çarpı d; örneğin 32 çarpı 2048 çarpı 4096. Transformer'ın gördüğü giriş tam olarak bu. Bizim gpt-nano modelimizde V yalnızca 3, çünkü üç rakam var, ve d = 48.

12. Embedding lookup aslında bir matris çarpımıdır: one-hot vektör, tablonun tek bir satırını seçer

Kodda embedding genellikle embedding[token_id] diye, yani bir indeksleme olarak yazılır. Ama matematiksel olarak bunu bir matris çarpımı olarak da yazabiliriz ve bu yazım, embedding'in aslında ne olduğunu anlamak için önemli. Token ID'si i olsun. Bunun one-hot gösterimi, V uzunluğunda, yalnızca i'nci elemanı 1, diğer bütün elemanları 0 olan bir vektör.

Embedding, bu vektörün satır hâlinin E ile çarpımı: e eşittir x transpoz E. Boyutları kontrol edelim: 1 çarpı V'lik bir satır vektörü ile V çarpı d'lik bir matrisin çarpımı 1 çarpı d; yani d boyutlu bir vektör. Neden bu çarpım tam olarak bir satır seçiyor? Çarpımı açarsak, E'nin satırlarının x'in elemanlarıyla ağırlıklandırılmış toplamı. x'in yalnızca i'nci elemanı 1, diğerleri 0; dolayısıyla toplamdaki bütün terimler sıfırla çarpılıyor ve geriye yalnızca E'nin i'nci satırı kalıyor. Soldaki sayısal örnek bunu somut gösteriyor: V = 5, d = 3. Embedding matrisi 5 satır 3 sütun. 3. token için one-hot vektör [0 0 1 0 0]. Çarpımın sonucu [−0,4; 0,6; 0,2]; bu, E'nin tam olarak 3. satırı. Bu eşdeğerliği kendi modelimizde de test ettik: nn.Embedding'in çıktısı ile one-hot matris çarpımının çıktısı birebir aynı. O zaman kod neden çarpım değil de indeksleme kullanıyor? Maliyet yüzünden: çarpım olarak V çarpı d çarpma-toplama yapmamız gerekirdi, oysa indeksleme olarak yalnızca d tane sayı okuyoruz, hiç çarpma yok. Bu, embedding'in matematiksel olarak bir Linear katman olduğu, ama girişi one-hot olduğu için hesabının bir tablo okumasına indiği anlamına geliyor.

13. Neden one-hot'ta kalmıyoruz? One-hot vektörlerin hepsi birbirine eşit uzaklıktadır, benzerlik bilgisi taşımaz

Madem embedding one-hot vektörle bir çarpım, neden modele doğrudan one-hot vektörü vermiyoruz? Çünkü one-hot gösterimi hiçbir anlamsal bilgi taşımaz. Soldaki kutuya bakalım: kedi, köpek ve masa için üç one-hot vektör. Bunların her biri farklı bir eksen üzerinde duruyor. İki farklı one-hot vektör arasındaki Öklid uzaklığı her zaman √2, yaklaşık 1,41: kedi ile köpek arasında da, kedi ile masa arasında da aynı. Yani one-hot uzayında "kedi köpeğe masadan daha benzerdir" bilgisi temsil edilemez; bütün token'lar birbirinden eşit uzaklıkta. Embedding uzayı ise bunu öğrenebilir. Sağdaki kutu kavramsal bir örnek, üç boyutlu ve temsilî. Kedi ve köpek vektörleri neredeyse aynı yöne bakıyor; aralarındaki uzaklık 0,07, cosine benzerlikleri neredeyse 1. Masa ise başka bir yöne bakıyor; kedi ile masa arasındaki cosine 0, yani dikler. Böylece anlamsal benzerlik geometrik benzerliğe dönüşmüş oluyor. Alttaki kutu, benzerliği ölçmek için en çok kullanılan araç: cosine benzerlik. İki vektörün iç çarpımının uzunluklarının çarpımına bölümü; esasen aralarındaki açının kosinüsü. 1 aynı yön, 0 dik, eksi 1 tam ters yön demek. İyi bir embedding'in temel amacını tek cümleyle şöyle söyleyebiliriz: anlamsal yakınlık yaklaşık olarak vektörel yakınlık olmalı. Bunu bizim modelimizde de ölçeceğim; birkaç slayt sonra gpt-nano'nun öğrendiği embedding'lerin cosine benzerliklerini göstereceğim.

14. Embedding öğrenilmiş bir koordinat sistemidir: eksenlerin anlamını biz değil, training belirler

Embedding'in bence en önemli yanı şu: embedding, öğrenilmiş bir koordinat sistemi. Model kelimelere bizim önceden belirlediğimiz canlılık, cinsiyet, büyüklük, fiil olma gibi özellikler atamıyor; eğitim sırasında uygun bir koordinat sistemi kendiliğinden oluşuyor. Bunu

anlamak için oyuncak bir örnek düşünelim: üç boyutlu bir embedding ve eksenlerin kabaca canlılık, insanlık ve mekân anlamına geldiğini varsayalım. O zaman kedi ve köpek canlılık ekseninde yüksek, diğerlerinde düşük; doktor hem canlı hem insan; ev ise mekân ekseninde. Bu vektörlerden cosine hesaplarsak kedi ile köpek neredeyse 1, kedi ile doktor 0,76, kedi ile ev 0,11 çıkıyor. Bu sayıları gerçekten hesapladık. Ama dürüst olmak gerekirse gerçek modeller böyle temiz değil. Gerçek LLM'lerde boyut 2048, 4096, 8192 gibi değerler ve özellikler tek tek eksenlere dağılmıyor. Buna distributed representation deniyor: "canlılık" gibi bir kavram tek bir boyutta tutulmuyor, birçok boyutun bir karışımı olarak, yani uzayda bir yön olarak temsil ediliyor. Bu yüzden "4096 boyut, 4096 kavram" demek yanlış. Yönlerin anlam taşıyabildiğinin en ünlü örneği Word2Vec'ten: king vektöründen man'i çıkarıp woman'ı eklediğinizde queen'e yakın bir noktaya geliyorsunuz. Yani "erkekten kadına" gibi bazı ilişkiler uzayda yaklaşık sabit bir yön olarak kodlanıyor. Modern LLM'lerde temsillerin hepsi bu kadar basit doğrusal aritmetikle açıklanamıyor, ama sezgi değerli. Son olarak d neden bu kadar büyük? Çünkü doğal dilde çok sayıda gizli özellik var ve bunların kombinasyonlarını temsil etmek için geniş bir uzay gerekiyor.

15. Embedding nasıl öğrenilir? Gradient yalnızca dizide geçen token'ların satırlarına gider

Embedding tablosunun değerleri nereden geliyor? Training ile öğreniliyor. Bunun matematiğine bakalım, çünkü ilginç bir özelliği var. t pozisyonundaki giriş vektörü, W_{te}'nin transpozu ile o pozisyondaki token'ın one-hot vektörünün çarpımı, artı position tablosunun t'nci satırı. Loss'un bu vektöre göre gradient'ini, yani $\partial L/\partial x_t$'yi geri yayılım bize veriyor. Zincir kuralıyla W_{te}'ye göre gradient, her pozisyon için one-hot vektör ile o pozisyonun gradient'inin dış çarpımlarının toplamı oluyor. One-hot vektörün yalnızca bir elemanı 1 olduğu için, bu dış çarpım yalnızca bir satırı dolduruyor. Sonuç ikinci satırdaki formül: W_{te}'nin i'nci satırının gradient'i, i numaralı token'ın geçtiği bütün pozisyonlardaki gradient'lerin toplamı. Sağdaki tablo bunu somutlaştırıyor: 2 0 1 1 0 2 girdisinde, 0 token'ının satırı 1. ve 4. pozisyonlardan gradient topluyor; 1 token'ının satırı 2. ve 3. pozisyonlardan; 2 token'ının satırı 0. ve 5. pozisyonlardan. Bir token bir batch'te hiç geçmiyorsa satırına hiç gradient gitmiyor; buna sparse gradient denir. Büyük vocabulary'li modellerde nadir token'ların satırlarının yavaş öğrenilmesinin sebebi bu. İkinci önemli nokta: x bir toplam olduğu için, aynı gradient hem token satırına hem position satırına olduğu gibi gidiyor; toplama işleminin türevi bir. Başlangıç değerleri ve düzenlileştirme de koddan okunabiliyor: minGPT bütün embedding elemanlarını ortalaması sıfır, standart sapması 0,02 olan normal dağılımdan çekiyor. Eğittiğimiz modelde training sonunda standart sapmalar 0,049 ve 0,060'a çıkmış; yani tablolar başlangıç noktasından anlamlı biçimde uzaklaşmış. Bir ayrıntı daha: minGPT, weight decay'i yalnızca Linear katmanların weight'lerine uyguluyor; LayerNorm ve embedding parametreleri "blacklist"te, yani

embedding'ler sıfıra doğru çekilmiyor. Bunun mantığı, embedding'in bir dönüşüm değil bir temsil olması; onu küçültmek bilgiyi kaybettirir.

16. Gerçek modelde: gpt-nano'nun position embedding'leri, girdi ve çıktı bölgelerini kendiliğinden ayırmış

Embedding'in matematiğini anlattıktan sonra, bizim eğittiğimiz gpt-nano'nun öğrendiği tablolara bakalım. Bu slayttaki bütün sayılar modelin kendisinden. Tablo, 11 position embedding vektörünün ikişerli cosine benzerliği. Cosine benzerlik, iki vektörün iç çarpımının uzunluklarının çarpımına bölümü; 1 aynı yön, 0 dik, eksi 1 tam ters yön demek. Turuncu hücreler aynı yönü, mavi hücreler ters yönü gösteriyor. 0'dan 5'e kadar olan pozisyonlar girdi rakamları, 6'dan 10'a kadar olanlar modelin yazdığı çıktı rakamları. Üç yapı hemen göze çarpıyor ve hiçbirini biz programlamadık. Birincisi: girdi bölgesindeki 1'den 4'e kadar olan pozisyonlar birbirine çok benziyor, cosine 0,64 ile 0,85 arasında. Bu mantıklı, çünkü sıralama görevinde bir rakamın girdide hangi sırada geldiği sonucu değiştirmiyor; model girdi pozisyonlarını neredeyse eşdeğer görmeyi öğrenmiş. İkincisi: girdi pozisyonları ile çıktı pozisyonları zıt yönlü, cosine eksi 0,40 ile eksi 0,72 arasında. Yani model embedding uzayında "girdi bölgesi" ve "çıktı bölgesi" diye iki ayrı yön oluşturmuş. Üçüncüsü: çıktı bölgesinde komşu pozisyonlar birbirine benziyor, 0,54 ile 0,66 arası; model çıktıda kaçıncı rakamı yazdığını bu sürekli yapı ile takip ediyor. Vektör uzunlukları da bir şey söylüyor: girdi pozisyonlarında 0,16–0,20 civarında, çıktı pozisyonlarında 0,46–0,72. Çıktı tarafında pozisyon bilgisi çok daha baskın; çünkü orada model, sırada hangi rakamın geleceğini pozisyona göre belirlemek zorunda. Token embedding'lerine bakınca da üç token'ın birbirinden uzaklaştığını görüyoruz: 0 ile 1 arasında eksi 0,40, 0 ile 2 arasında eksi 0,50. 48 boyutlu bir uzayda üç vektör için bu, birbirini rahatça ayırt edebilecek bir düzen. Sonuç: embedding tabloları rastgele sayılar değil; görevin yapısını geometrik olarak kodluyorlar.

17. Pozisyon bilgisi üç yolla verilir: learned tablo, sinusoidal formül ya da RoPE ile döndürme

Embedding'in ikinci yarısı pozisyon bilgisi. Bunu vermenin üç yaygın yolu var ve minGPT bunların en basitini kullanıyor. Birincisi learned position embedding, minGPT ve GPT-2'nin yolu: her pozisyon için ayrı bir öğrenilen vektör, token vektörüne ekleniyor. T çarpı C parametre gerekiyor. Zayıf yanı: tabloda yalnızca T satır olduğu için, model training'de görmediği daha uzun bir context'e genelleşemez. İkincisi, orijinal Transformer makalesindeki sinusoidal kodlama. Burada tablo öğrenilmiyor, bir formülle hesaplanıyor: her boyut çifti için, pozisyon t'ye bağlı bir sinüs ve kosinüs; her çiftin frekansı farklı. Bunu, farklı hızlarda dönen saatler gibi düşünebiliriz: düşük boyutlar hızlı, yüksek boyutlar yavaş dönüyor. Parametresi yok ve pozisyon k kadar kaydırıldığında her çift sabit bir açıyla dönüyor, yani doğrusal bir dönüşümle elde ediliyor. Üçüncüsü RoPE, Llama ve nanochat'in

yolu. Burada pozisyon bilgisi x'e eklenmiyor; attention içinde q ve k vektörlerine uygulanıyor. q'nun her boyut çifti, t çarpı θ_i açısıyla döndürülüyor. Alttaki kutu RoPE'nin neden çalıştığının ispatı ve tek satır. t pozisyonundaki query'yi tθ kadar, s pozisyonundaki key'i sθ kadar döndürdüğümüzü düşünelim. Dot product'ları, q transpoz, R(tθ) transpoz, R(sθ), k. Döndürme matrisleri ortogonal olduğu için R(a) transpoz R(b), R(b eksi a)'ya eşit. Dolayısıyla score yalnızca q'ya, k'ya ve iki pozisyon arasındaki farka, s eksi t'ye bağlı. Mutlak pozisyon kayboluyor, yalnızca göreli uzaklık kalıyor. Bu, dil için doğal bir özellik: iki kelimenin ilişkisi metnin neresinde durduklarından çok, aralarındaki mesafeye bağlı. Donanım açısından fark net: minGPT'nin learned position'ı bir tablo okuması ve bir toplama; RoPE ise her token'da, her head için, her boyut çiftinde kosinüs ve sinüs değerleriyle çarpma gerektiriyor.

18. Token embedding sabittir; layer'lar ilerledikçe aynı token'ın vektörü bağlama göre değişir

Embedding kelimesi LLM konuşmalarında iki, hatta üç farklı şeyi ifade edebiliyor; bu ayrımı netleştirmek istiyorum. Birincisi token embedding: Transformer'a girmeden önceki vektör, yani E matrisinin bir satırı. Bu sabit. "bank" token'ının embedding tablosundaki satırı, hangi cümlede geçerse geçsin aynı. Ama "bank" İngilizcede hem para yatırılan banka hem de nehir kıyısı demek. Model bu iki anlamı nasıl ayırıyor? Cevap formüllerde. İlk temsil, token embedding artı pozisyon bilgisi. Sonra her layer bir öncekinin çıktısını alıp yeni bir temsil üretiyor: H l artı bir, F_l'nin H l'ye uygulanması. L layer sonra i'nci token'ın temsili artık yalnızca o token'a değil, bütün cümleye bağlı bir fonksiyon. Bunun mekanizması attention: yeni temsil z_i, diğer token'ların value'larının ağırlıklı bir toplamı. Yani Transformer katmanlarında bir token'ın vektörü kabaca "token kimliği artı bağlam bilgisi" hâline geliyor. Bu yüzden iki cümlede "bank" başlangıçta aynı vektörle başlıyor ama son layer'da farklı vektörlerle temsil ediliyor. Buna contextual representation deniyor ve LLM'lerin gücünün çok önemli bir parçası. Üçüncü anlam, RAG sistemlerinde kullanılan sentence embedding. Orada ayrı bir embedding modeli bütün bir cümleyi ya da dokümanı tek bir vektöre çeviriyor; sorgu vektörü ile doküman vektörleri cosine benzerlikle karşılaştırılıp en yakın dokümanlar bulunuyor. Bu, vector search'ün mantığı. Ama bu, bizim konuştuğumuz token embedding ile aynı şey değil. Bu sunumda "embedding" dediğimde hep birincisini kastediyorum.

19. Adım 2 · LayerNorm, her token vektörünü ortalaması 0 ve varyansı 1 olacak şekilde yeniden ölçekler

İkinci adım LayerNorm. Her token'ın vektörünü kendi içinde normalize ediyoruz: vektörün elemanlarının ortalamasını çıkarıyor, standart sapmasına bölüyoruz. Böylece her vektörün ortalaması sıfır, varyansı bir oluyor. Ardından öğrenilen iki parametre uygulanıyor: γ ile

eleman başına çarpıyor, β ile kaydırıyoruz; yani model gerekirse normalizasyonun etkisini kısmen geri alabiliyor. Neden gerekli? Derin bir ağda vektörler her layer'dan geçtikçe büyüklükleri kontrolsüz biçimde büyüyüp küçülebilir. Bu da training'i kararsız yapar. LayerNorm, her alt adımın girişinde ölçeği sabitliyor. Sağdaki küçük örnek: [2, 4, 6] vektörünün ortalaması 4, standart sapması yaklaşık 1,63. Ortalamayı çıkarıp standart sapmaya bölünce yaklaşık [−1,22, 0, 1,22] oluyor; çubuklar sıfır çizgisinin iki yanına simetrik dağılıyor. minGPT, GPT-2 gibi "pre-LN" düzenini kullanıyor: LayerNorm, attention'dan ve MLP'den önce uygulanıyor. Koddaki son iki satır bunu gösteriyor: önce ln_1, sonra attention; önce ln_2, sonra MLP. Dikkat edilecek nokta, residual yolun kendisinin normalize edilmemesi; normalize edilen kopya alt adıma giriyor, sonuç orijinal x'e ekleniyor. FPGA açısından bir not düşmek istiyorum: LayerNorm'da ortalama, karekök ve bölme var. Toplama ve çarpma donanımda ucuz ama karekök ve bölme pahalı ve yavaş. Bu adımı FPGA potansiyeli bölümünde tekrar anacağım.

20. Attention neden gerekli? Her token'ın, önceki token'lardan bilgi toplaması gerekiyor

Sıra modelin en önemli parçasına, attention'a geldi. Formüle geçmeden önce hangi problemi çözdüğünü anlatmak istiyorum. Embedding'ten sonra her pozisyonda bir vektörümüz var, ama bu vektör yalnızca kendi token'ını ve pozisyonunu biliyor. Bir sonraki token'ı tahmin etmek için ise geçmişe bakmak gerekiyor. Sıralama görevinde çıktıyı yazarken model girdideki rakamlara bakmak zorunda; bir isimde de sonraki harf, önceki harflere bağlı. Yani her pozisyonda geçmişteki vektörlerden bilgi toplayıp tek bir vektörde birleştirmemiz gerekiyor. Diyelim ki dördüncü pozisyondayız ve elimizde x0, x1, x2, x3 var. Bunlardan tek bir y3 vektörü nasıl elde ederiz? En basit fikir, üstteki kutu: hepsinin ortalamasını almak. Her birine 0,25 ağırlık. Bu bir şey öğrenir ama ciddi bir sınırı var: model neye bakacağını seçemez; önemli token da önemsiz token da aynı ağırlığı alır. Attention'ın fikri alttaki kutu: yine bir ağırlıklı ortalama alıyoruz, ama ağırlıklar sabit değil. Model, token'ların içeriğine bakarak hangisinin şu an önemli olduğuna kendisi karar veriyor. Örnekte ağırlığın 0,70'i x1'e gidiyor; çünkü model x1'in bu tahmin için en ilgili token olduğuna karar vermiş. Değerler temsilî, ama birazdan bizim modelimizden gerçek değerler göstereceğim. Alttaki tanımı vurgulamak istiyorum, çünkü bütün attention bu tek cümleden ibaret: attention, geçmişteki vektörlerin, ağırlıkları içeriğe göre hesaplanan bir ağırlıklı ortalamasıdır. Ağırlıklar her zaman pozitif ve toplamları bir. Geri kalan her şey, yani query, key, value, softmax, bu ağırlıkların nasıl hesaplandığıyla ilgili.

21. Attention'ın fikri: bir sözlükte arama yapmanın "yumuşak" bir versiyonu

Ağırlıkları içeriğe göre nasıl hesaplarız? Bunun için en iyi benzetme bir sözlükte arama yapmak. Soldaki kutu sıradan bir Python dict'i. Her key'in bir value'su var. d["kedi"] dediğimizde aradığımız şey, yani query, tam olarak bir key ile eşleşiyor ve yalnızca o key'in value'su dönüyor. Eşleşen key'in ağırlığı 1, diğerlerinin 0. Attention bu aramanın yumuşak bir versiyonu. Sağdaki kutuya bakalım. Query yine var, key'ler ve value'lar yine var; ama tam eşleşme aramıyoruz. Query'yi her key ile karşılaştırıp bir benzerlik puanı hesaplıyoruz; bu puan iki vektörün dot product'ı. Sonra bu puanları softmax ile pozitif ve toplamı 1 olan ağırlıklara çeviriyoruz. Sonuç, value'ların bu ağırlıklarla alınmış karışımı. Örnekte query en çok k0'a benziyor, ağırlığın 0,70'i v0'a gidiyor; ama v1 ve v2 de az da olsa katkı yapıyor. Yumuşak aramanın büyük avantajı türevlenebilir olması: sert aramada "hangi key" sorusunun türevi yok, ama yumuşak aramada her ağırlık sürekli değişiyor. Bu yüzden model, training sırasında neye bakacağını gradient ile öğrenebiliyor. Alttaki üç kutu üç kavramı özetliyor. Query, şu anki token'ın "ne arıyorum" sorusu. Key, her token'ın "bende ne var" diye kendini tanıtan etiketi. Value ise o token seçilirse aktaracağı içerik. Kodun üç satırı da attention'ın neredeyse tamamı. Bir sonraki slaytta bu üç vektörün nereden geldiğini, sonra da gerçek formülü göstereceğim.

22. Adım 3 · Query, key ve value aynı vektörden, öğrenilen üç linear projection ile üretilir

Üçüncü adım: query, key ve value vektörlerini üretmek. Bunlar dışarıdan gelmiyor; her token kendi vektöründen üretiyor. Normalize edilmiş 48 boyutlu vektör, c_attn adlı bir Linear katmandan geçiyor. Bu katmanın 144 çıkışı var; bunlar 48'erli üç parçaya bölünüyor: birinci parça query, ikinci key, üçüncü value. Yani aslında üç ayrı öğrenilmiş projection var, minGPT verimlilik için bunları tek bir matriste birleştirmiş. Sonuçta her token'ın kendi query'si, key'i ve value'su oluyor. Query, o token'ın geçmişte ne aradığını; key, diğer token'lara kendini nasıl tanıttığını; value ise seçilirse ne aktaracağını temsil ediyor. Bu isimler bir benzetme; matematikte yalnızca üç öğrenilmiş linear projection var ve hangi rolü üstlenecekleri training sırasında ortaya çıkıyor. Burada Linear katmanı da tanımlamak istiyorum, çünkü modelin her yerinde karşımıza çıkacak. Linear katman bir matris-vektör çarpımı artı bir bias. Her çıkış elemanı, girişlerin öğrenilmiş weight'lerle ağırlıklı toplamı. Bu tek satır, her token için 144 çarpı 48, yani 6 912 çarpma-toplama yapıyor. Çarpma-toplamaya kısaca MAC diyoruz: multiply-accumulate. Modelin hesabının büyük çoğunluğu bu tür Linear katmanlar; bunu FPGA bölümünde sayılarla göstereceğim, çünkü bir donanım tasarımının neye odaklanması gerektiğini tam olarak bu belirliyor. Kodda iki satır var: tanımda tek bir Linear, forward'da çıkışın split ile üçe bölünmesi.

23. Adım 4 · Scaled dot-product attention üç işlemdir: score, softmax ve value'ların ağırlıklı toplamı

Şimdi attention'ın gerçek formülü. Adı scaled dot-product attention ve üç işlemden oluşuyor. Birinci işlem score. t pozisyonundaki token'ın query'sini, geçmişteki her j pozisyonunun key'i ile dot product'a sokuyoruz. İki vektör aynı yöne bakıyorsa dot product büyük, farklı yönlere bakıyorsa küçük ya da negatif. Yani score, "bu geçmiş token, şu an aradığım şeye ne kadar benziyor" sorusunun cevabı. j küçük eşit t koşulu, yalnızca geçmişe bakıldığını söylüyor; bunu birazdan maske ile göreceğiz. Score'u $\sqrt{d_h}$'ye, yani head boyutunun kareköküne bölüyoruz. Bunun nedeni şu: q ve k'nin bileşenleri yaklaşık bağımsız ve varyansları bir ise, dot product d_h tane çarpımın toplamı olduğu için varyansı d_h olur. Head boyutu büyüdükçe score'lar büyür; softmax üstel bir fonksiyon olduğu için büyük farklar dağılımı tek bir token'a kilitler ve gradient'ler söner. $\sqrt{d_h}$'ye bölmek varyansı bire sabitliyor. İkinci işlem softmax: score'ların üstelini alıp toplamlarına bölüyoruz. Böylece her score pozitif bir weight'e dönüşüyor ve weight'lerin toplamı bir oluyor. Büyük score, büyük weight demek. Üçüncü işlem ağırlıklı toplam: value vektörlerini bu weight'lerle çarpıp topluyoruz. Sonuç y, bu token'ın geçmişten topladığı bilgi. Alttaki kod minGPT'den aynen alındı ve her satır bir formüle karşılık geliyor. İlk satır q ile k'nin transpozunun çarpımı ve ölçekleme; yani bütün score'lar tek bir matris çarpımıyla. İkinci satır maske; geleceği eksi sonsuz yapıyor. Üçüncü satır softmax. Dördüncü satır weight'lerle v'nin çarpımı. Kodun bütün diziyi aynı anda, matris çarpımlarıyla işlediğine dikkat: bu GPU için çok verimli. Donanımda ise generation sırasında her adımda tek bir token işleyeceğiz; bunu KV cache slaytında açacağım.

24. Elle bir örnek: üç geçmiş token, $d_h = 2$ ve x yönünde bir şey arayan bir query

Formülü somutlaştırmak için elle küçük bir örnek yapalım. Head boyutu 2 olsun ki her şeyi kafadan hesaplayabilelim. Şu anki token'ın query'si [1, 0]; yani x ekseni yönünde bir şey arıyor. Geçmişte üç token var. Birincinin key'i [2, 0], tamamen x yönünde. İkincinin key'i [0, 2], tamamen y yönünde. Üçüncünün key'i [1, 1], ikisinin arasında. Birinci işlem score: query'yi her key ile dot product'a sokuyoruz. [1, 0] ile [2, 0]'ın dot product'ı 2; [0, 2] ile 0; [1, 1] ile 1. Bunları $\sqrt{2}$'ye bölüyoruz, çünkü head boyutu 2: 1,41, 0 ve 0,71 elde ediyoruz. İkinci işlem softmax: her birinin üstelini alıyoruz; 4,11, 1 ve 2,03. Toplamları 7,14. Her birini toplama bölünce weight'ler çıkıyor: 0,58, 0,14 ve 0,28. Toplamları tam olarak 1. Üçüncü işlem ağırlıklı toplam: value'ları bu weight'lerle çarpıp topluyoruz. Sonuç [0,86, 0,42]. Bu sonucu yorumlamak önemli. Query x yönünü arıyordu; en çok x yönlü key'e sahip birinci token'a baktı, ona 0,58 weight verdi. y yönlü ikinci token'a en az baktı, 0,14. Ortadaki üçüncü token ara bir pay aldı. Sonuç vektörü de ağırlıklı olarak birinci token'ın value'sunu taşıyor ama diğerlerinden de pay almış. İşte bir token'ın geçmişten "içeriğe göre" bilgi toplaması tam olarak bu. Gerçek modelde vektörler 16 boyutlu ve weight'ler training ile

öğreniliyor, ama hesap bire bir aynı. Ayrıca bu tablo, attention'ın hesap maliyetini de gösteriyor: her geçmiş token için bir dot product, bir üstel, bir de value ile çarpma. Geçmiş uzadıkça bu iş doğrusal olarak artıyor.

25. Causal mask: her token yalnızca kendisine ve geçmişine bakabilir, gelecek tamamen gizlenir

Formülde j küçük eşit t koşulu vardı. Bu koşulu sağlayan mekanizmaya causal mask diyoruz. Soldaki tablo bütün query ve key pozisyon çiftlerini gösteriyor. Satırlar query pozisyonu, yani "şu an hangi token'dayım"; sütunlar key pozisyonu, yani "hangi token'a bakıyorum". Gri hücreler gelecek: j, t'den büyük. Bu hücrelerin score'u eksi sonsuz yapılıyor. Renkli hücreler ise softmax sonrası weight'ler; tonlar temsilî, gerçek değerler modele göre değişir. Her satır kendi içinde toplamı 1 olan bir dağılım. Neden böyle bir maske gerekli? Training sırasında model her pozisyonda bir sonraki token'ı tahmin etmeye çalışıyor. Eğer gelecekteki token'ları görebilseydi, cevap zaten önünde olurdu; kopyalardı ve hiçbir şey öğrenmezdi. Maske tam olarak nasıl çalışıyor? Score eksi sonsuz yapılınca softmax'taki üstel değer sıfır oluyor. Yani maskelenen weight yaklaşık olarak değil, tam olarak sıfır. Bunun güzel bir yan etkisi var: her pozisyon yalnızca kendi geçmişini gördüğü için, bir dizideki bütün pozisyonların tahmini tek bir forward geçişinde, birbirini bozmadan, paralel olarak yapılabiliyor. minGPT'nin training'i bu yüzden verimli. Ve en önemli sonuç alttaki cümle: t pozisyonundaki çıktı yalnızca 0'dan t'ye kadar olan token'lara bağlı. Yeni bir token geldiğinde eski pozisyonların hiçbir değeri değişmiyor. Bu, birkaç slayt sonra anlatacağım KV cache'in, yani generation'ı ucuzlatan fikrin matematiksel temeli. Kodda maske, torch.tril ile oluşturulan bir alt üçgen matris; bir kez oluşturulup modelin içinde saklanıyor.

26. Gerçek modelde: gpt-nano'nun bir head'i, çıktıyı yazarken girdideki ilgili rakamlara bakıyor

Şimdiye kadarki örnekler temsilîydi. Bu slayttaki sayılar ise gerçek: bizim eğittiğimiz gpt-nano modeline 2 0 1 1 0 2 girdisini verdik ve birinci layer'ın ilk head'inin softmax weight'lerini çıkardık. Tablonun her satırı bir pozisyon. Satırdaki sayılar, o pozisyonda bir sonraki token tahmin edilirken geçmişteki her pozisyona ne kadar weight verildiğini gösteriyor. Sağ üst üçgen gri, çünkü causal mask geleceği kapatıyor. Lacivert etiketler girdi token'ları, turuncu olanlar modelin ürettiği çıktı token'ları. İki satıra dikkatinizi çekmek istiyorum. Beşinci pozisyon girdinin son rakamı; burada model ilk çıktıyı, yani en küçük rakamı tahmin etmek zorunda. Bu head ağırlığının yüzde 96'sını girdideki iki 0'a veriyor: 1. pozisyondaki 0'a 0,36, 4. pozisyondaki 0'a 0,60. Model "en küçük rakamı yazacağım, girdide 0 var mı?" diye bakıyor gibi. Onuncu pozisyonda ise model son çıktıyı, yani en büyük rakamı tahmin ediyor. Bu kez ağırlığın yüzde 99'u girdideki iki 2'de: 0. pozisyon 0,31, 5. pozisyon 0,68. İşte attention'ın "içeriğe göre ağırlıklı ortalama" olmasının pratikteki anlamı bu: model

hangi token'a bakacağını, o an ne yapması gerektiğine göre seçiyor. Önemli olan şu: bu davranışı biz programlamadık. Modele yalnızca sıralanmış örnekler gösterdik; bu bakma deseni training sırasında kendiliğinden ortaya çıktı. Dürüst olmak için şunu da söyleyeyim: diğer head'lerin desenleri bu kadar net değil. Sıralama işi tek bir head'de değil, üç layer ve dokuz head arasında paylaşılıyor; bu head en kolay okunanı.

27. Adım 5 · Multi-head attention: 48 boyut üç head'e bölünür ve her head kendi ilişkisini arar

Beşinci adım: multi-head attention. Tek bir attention, her pozisyonda tek bir ağırlıklı ortalama alabiliyor. Ama bir token'ın aynı anda birden çok şeye bakması gerekebilir. Örneğin sıralamada hem girdideki rakamları saymak hem de şimdiye kadar kaç rakam yazıldığını bilmek gerekiyor. Çözüm, attention'ı birkaç kez paralel çalıştırmak. minGPT bunu şöyle yapıyor: 48 boyutlu q, k ve v vektörlerini 16'şar boyutluk üç parçaya bölüyor. Her parça bir head. Her head, kendi 16 boyutlu q, k ve v'siyle kendi score'larını, kendi softmax weight'lerini ve kendi ağırlıklı toplamını bağımsız olarak hesaplıyor. Az önce gösterdiğim, girdideki rakamlara bakan head bunlardan biriydi. Sonra üç head'in 16 boyutlu çıktıları yan yana konuyor, yine 48 boyut oluyor ve c_proj adlı bir Linear katman bunları karıştırıyor. Böylece her head'in bulduğu bilgi tek bir vektörde birleşiyor. Maliyet açısından önemli bir nokta var: multi-head, hesabı artırmıyor. Toplam çarpma sayısı tek head'li, 48 boyutlu bir attention ile aynı; çünkü vektör yalnızca parçalara bölünüyor. Kodda bu, view ve transpose ile yapılan bir yeniden şekillendirme. view, 48 boyutu 3 çarpı 16 olarak yeniden yorumluyor; transpose da head boyutunu öne alıyor. Böylece PyTorch, head'leri sanki ayrı örneklermiş gibi tek bir matris çarpımında işliyor. İşlem bitince ters yönde birleştirip c_proj'dan geçiriyor. minGPT kodunda nh yerine self.n_head yazıyor; okunabilirlik için kısalttım.

28. Adım 6 · Residual connection: alt adımlar x'i silmez, üstüne bir düzeltme ekler

Altıncı adım residual connection. Bir block'un kodunun tamamı sağda: dört satır. Dikkat edilecek nokta, attention'ın ve MLP'nin sonucunun x'in yerine geçmemesi; x'in üstüne eklenmesi. Şekil bunu gösteriyor. x soldan sağa akan bir "residual stream". Attention, bu stream'in normalize edilmiş bir kopyasını okuyor, bir sonuç üretiyor ve sonucu stream'e ekliyor. Ardından MLP aynı şeyi yapıyor. Her block'ta iki ekleme var; üç block'lu modelde toplam altı. Bunun iki önemli sonucu var. Birincisi training ile ilgili. Formülde görüldüğü gibi, x artı f(x)'in türevi birim matris artı f'nin türevi. Yani türevde her zaman bir birim matris kalıyor. Bu sayede gradient, derin bir ağda her katmanda çarpılarak küçülmek yerine bu "kısa yol" üzerinden doğrudan geri akabiliyor. Residual connection olmadan derin transformer'ları eğitmek çok zor; bu fikir aslında görüntü işlemedeki ResNet'ten geliyor. İkincisi yorum ile ilgili. Residual stream'i bütün layer'ların ortak çalışma alanı gibi

düşünebiliriz. Embedding bu alana token ve pozisyon bilgisini yazıyor. Her attention ve her MLP buradan okuyor ve buraya küçük bir düzeltme yazıyor. lm_head en sonda bu alanın son hâlini okuyor. Bu bakış açısı modelin ne yaptığını anlamak için çok kullanışlı. Donanım açısından da bir not: residual connection yalnızca bir toplama; ucuz bir işlem. Ama bu stream bütün model boyunca taşındığı için, sayıların hangi hassasiyette tutulacağı önemli bir tasarım kararı oluyor.

29. Adım 7 · MLP her token'ı kendi içinde işler: önce dört kat genişletir, GELU'dan geçirir, sonra daraltır

Yedinci adım MLP. Attention token'lar arasında bilgi taşıyordu; MLP ise her token'ın vektörünü kendi içinde, diğer token'lara bakmadan işliyor. Yapısı üstteki şeritte: normalize edilmiş 48 boyutlu vektör önce c_fc adlı Linear ile 192 boyuta, yani dört katına genişletiliyor. Sonra her elemana GELU adlı doğrusal olmayan fonksiyon uygulanıyor. Sonra c_proj adlı ikinci Linear ile tekrar 48 boyuta indiriliyor ve sonuç residual stream'e ekleniyor. Neden ortada doğrusal olmayan bir fonksiyon var? Çünkü iki Linear katman arka arkaya gelirse, matematiksel olarak tek bir Linear katmana eşittir: iki matrisin çarpımı yine bir matristir. Ortada bir non-linearity olmazsa genişletmenin hiçbir anlamı kalmaz. Modelin karmaşık ilişkileri temsil edebilmesi bu fonksiyonlar sayesinde. Sağdaki grafik GELU'yu gösteriyor; kesikli mavi çizgi karşılaştırma için ReLU. İkisi de pozitif değerleri neredeyse olduğu gibi geçiriyor, negatifleri sıfıra yakın bastırıyor. GELU'nun farkı, köşesinin yumuşak olması ve negatif tarafta küçük bir çukur bırakması; bu, türevin her yerde düzgün olmasını sağlıyor. minGPT, GPT-2 ile aynı tanh yaklaşımlı formülü kullanıyor; formül biraz karmaşık görünüyor ama tek girişli bir fonksiyon. Bu, donanım için önemli bir avantaj: tek girişli bir fonksiyon, giriş değerleri sınırlıysa önceden hesaplanmış bir tabloyla temsil edilebilir. Kodda MLP tek satırlık bir lambda: c_fc, GELU, c_proj ve dropout sırayla uygulanıyor. Bir de parametre sayısı açısından not: MLP'nin iki Linear'ı, her block'taki parametrelerin yaklaşık üçte ikisini oluşturuyor.

30. Adım 8 · Son LayerNorm ve lm_head her token için bir logit üretir; generation bunlardan birini seçer

Son adım çıkış. Block'lardan çıkan vektör son bir LayerNorm'dan, ln_f'den geçiyor; ardından lm_head adlı Linear katman vocabulary'deki her token için bir skor üretiyor. Bu skorlara logit diyoruz. Sıralama görevinde vocabulary yalnızca üç token, 0, 1 ve 2; dolayısıyla üç logit çıkıyor. Logit'ler softmax'tan geçince toplamı bir olan olasılıklara dönüşüyor. Örnekte logit'ler 2,2, 0,4 ve eksi 0,8; olasılıklar 0,82, 0,14 ve 0,04. Değerler temsilî. Şimdi bu dağılımdan bir token seçmemiz gerekiyor. İki yol var. Birincisi greedy decoding: her zaman en olası token'ı seç. Burada önemli bir gözlem var: softmax monoton artan bir fonksiyon, yani sıralamayı değiştirmiyor. En büyük logit her zaman en büyük olasılığa

karşılık geliyor. Bu yüzden greedy decoding'de softmax'ı hiç hesaplamadan doğrudan en büyük logit'i seçebiliriz. Donanım için bu güzel bir kolaylık, çünkü softmax'ta üstel fonksiyon ve bölme var. İkincisi sampling: dağılımdan rastgele çekmek. Bu durumda yüzde 82 ihtimalle 0, yüzde 14 ihtimalle 1 seçilir. Metin üretiminde çeşitlilik için sampling, sıralama gibi tek doğru cevabı olan görevlerde greedy kullanılıyor. Sağdaki kod minGPT'nin generate fonksiyonunun döngü gövdesi. Model çağrılıyor, son pozisyonun logit'leri alınıyor, temperature ile ölçekleniyor, softmax uygulanıyor, sonra sampling ya da greedy ile bir token seçilip dizinin sonuna ekleniyor. İlk satıra özellikle dikkat çekmek istiyorum: self(idx_cond). minGPT her yeni token için bütün diziyi, yani bütün prefix'i baştan modelden geçiriyor. Yüzüncü token'ı üretirken önceki doksan dokuz token'ı da yeniden hesaplıyor. Bu kodu basit tutuyor ama çok israflı. Çıkış tarafının matematiğine bir slayt daha ayırdıktan sonra, bunun neden matematiksel olarak gereksiz olduğunu göstereceğim.

31. Çıkış tarafı: unembedding; her logit, son hidden state ile bir token vektörünün dot product'ıdır

Embedding'in bir de çıkış tarafında aynası var. Üstteki zincir bir LLM'in en kaba matematiksel resmi: ayrık token'lar embedding ile sürekli bir vektör uzayına taşınıyor; Transformer bu uzayda, T çarpı d boyutlu matrisleri yine T çarpı d boyutlu matrislere dönüştürerek hesap yapıyor; sonunda unembedding ile vocabulary uzayına geri dönülüyor ve softmax bir olasılık dağılımı veriyor. Ayrık semboller, sürekli hesap, sonra yine ayrık bir dağılım. Bence embedding'i anlamanın en doğru zihinsel modeli bu. Çıkıştaki matrise unembedding matrisi deniyor: V satır d sütun. Logit vektörü z, bu matris ile son hidden state h'nin çarpımı. Bunu satır satır yazarsak çok güzel bir yorum çıkıyor: i'nci token'ın logit'i, unembedding matrisinin i'nci satırı ile h'nin dot product'ı. Yani her token'ın çıkış tarafında da bir vektörü var ve model aslında şu soruyu çözüyor: "şu anki hidden state hangi token'ın yönüyle en uyumlu?" Örneğin "Paris is the capital of" cümlesinden sonra hidden state, France token'ının vektörüyle aynı yöne bakıyorsa, o dot product büyük çıkıyor ve softmax'tan sonra France'ın olasılığı yüksek oluyor. Gerçek modellerde yapı daha karmaşık, ama bu sezgi çok faydalı. Bazı modellerde unembedding matrisi doğrudan embedding matrisinin kendisi; buna weight tying deniyor ve güzel bir simetri yaratıyor: aynı tablo hem token'ı uzaya taşıyor hem de uzaydan geri okuyor. GPT-2'nin orijinali ve llama2.c böyle. minGPT ise lm_head'i ayrı ve bias'sız bir Linear katman olarak tutuyor; kod sağda.

32. KV cache: geçmiş token'ların key ve value'ları hiç değişmediği için saklanıp yeniden kullanılır

Bu slayt, matematikten donanıma geçişin anahtarı. minGPT her yeni token için bütün diziyi baştan hesaplıyordu. Peki gerçekten gerekli mi? Hayır, ve nedeni causal mask. Hatırlayın: maske yüzünden j pozisyonundaki çıktı yalnızca 0'dan j'ye kadar olan token'lara bağlı. Bunu

her layer için söyleyebiliriz; ispatı layer'lar üzerinden tümevarım. Embedding yalnızca o token'a ve pozisyona bakıyor. LayerNorm, Linear ve MLP her pozisyonda ayrı çalışıyor. Attention ise maske nedeniyle yalnızca kendinden önceki pozisyonların key ve value'larını okuyor; onlar da tümevarım hipotezine göre yalnızca kendi geçmişlerine bağlı. Sonuç: yeni bir token geldiğinde eski pozisyonların key ve value vektörleri hiç değişmiyor. O hâlde bunları her layer için saklayabiliriz; buna KV cache diyoruz. Şekil farkı gösteriyor. Soldaki tablo cache'siz durum: birinci adımda bir token, ikinci adımda iki token, beşinci adımda beş token baştan hesaplanıyor; toplam 15 token hesabı. Sağdaki tablo cache'li durum: her adımda yalnızca yeni token hesaplanıyor, eskilerin key ve value'ları cache'ten okunuyor; toplam 5. Fark, dizi uzadıkça karesel ile doğrusal arasındaki fark. Bunun donanım açısından çok önemli bir sonucu var. Her adımda tek bir token işlendiği için modeldeki her Linear katman, matris-matris çarpımı yerine bir matris-vektör çarpımına iniyor. Attention ise t+1 uzunluğunda iki dot product dizisine iniyor: query ile saklanmış key'ler ve weight'ler ile saklanmış value'lar. Sağdaki pseudo-code bunun özü: yeni key ve value'yu sakla, query'yi saklanmış bütün key'lerle karşılaştır, weight'lerle saklanmış value'ları topla. minGPT'nin kodunda bu yok, ama modern bütün LLM'ler generation sırasında bunu yapıyor. Bizim FPGA tarafındaki her şey bu tek-token-adımı üzerine kurulacak.

33. FPGA, işlevi üretimden sonra tasarlanan bir çiptir ve hesap için dört tür yapı taşı sunar

Son bölüme, minGPT'nin FPGA'de uygulanma potansiyeline geçiyorum. Önce FPGA'nin ne olduğunu kısaca hatırlatayım. Bir işlemci ya da GPU'nun devresi fabrikada sabitlenir; biz onlara yalnızca komut veririz. FPGA'de ise devrenin kendisini, çip üretildikten sonra biz tasarlarız. Bunun için dört tür yapı taşı var. LUT'lar 6 girişli küçük doğruluk tabloları; her türlü mantık fonksiyonu bunlarla kuruluyor. Flip-flop'lar tek bitlik register'lar; ara sonuçları tutmak ve hesabı aşamalara bölmek, yani pipeline kurmak için kullanılıyor. BRAM'ler çip içindeki bellek blokları; bizim çipte 140 tane 36 kilobitlik blok var, toplam yaklaşık 630 kilobayt. DSP48'ler hazır çarpma ve toplama birimleri; her biri her clock cycle'da bir çarpma-toplama, yani bir MAC yapabiliyor. Bu sayılar projede kullandığımız PYNQ-Z2 kartındaki Zynq XC7Z020 çipine ait; küçük ve ucuz bir çip. Zynq'in güzel yanı, aynı çipte bir ARM işlemci ile programlanabilir mantığın birlikte bulunması. İşlemci Linux çalıştırıyor; modeli yükleyip sonuçları okuyabiliyor. Ağır hesap ise programlanabilir mantıkta yapılıyor. GPU ile farkı şöyle özetleyebilirim: GPU genel amaçlı ve çok güçlü, ama sabit çekirdeklerde komut yürütüyor. FPGA'de ise veri yolunu, belleğin düzenini ve paralelliği tam olarak modelin ihtiyacına göre biçimlendirebiliyoruz. Bu esneklik, minGPT gibi yapısı tamamen bilinen ve sabit bir hesap için çok değerli. Şimdi minGPT'nin iş yükünün bu yapı taşlarına nasıl oturduğuna bakalım.

34. minGPT'nin iş yükü: bir token'ın hesabının %96–98'i matris-vektör çarpımı ve model çip içine sığıyor

Bir donanım tasarlamadan önce iş yükünü sayılarla bilmek gerekiyor. Bu tablo, KV cache kullanıldığında gpt-nano'nun bir token için yaptığı hesabı gösteriyor. Her block'ta dört Linear katman var. q, k, v projection'ı 144 çarpı 48, yani 6 912 çarpma-toplama. Attention çıkışındaki c_proj 2 304. MLP'nin genişletme ve daraltma katmanları 9 216'şar. Bir block toplam 27 648; üç block ve küçük lm_head ile birlikte token başına 83 088 MAC. Attention'ın kendi hesabı, yani score'lar ve ağırlıklı toplam, her layer'da 2 çarpı C çarpı t+1; pozisyona göre 96 ile 1 056 arasında değişiyor. Bu modelde context en fazla 11 token olduğu için attention toplamın yalnızca yüzde 2 ile 4'ü. Yani bir token'ın hesabının yüzde 96 ila 98'i matris-vektör çarpımı. Bu çok önemli bir bilgi: donanımın kalbi, hızlı bir matris-vektör çarpım birimi olmalı. DSP blokları tam olarak bunun için var. Sağdaki kutu bellek tarafı. gpt-nano'da 85,7 bin parametre var. Her biri 32 bitlik floating point olarak saklanırsa yaklaşık 343 kilobayt ediyor; bu bile çipin yaklaşık 630 kilobaytlık BRAM'ine sığıyor. 8 bite indirilirse 84 kilobayt; kapasitenin yalnızca yüzde 13'ü. Bunun anlamı şu: weight'ler için çipin dışındaki DRAM'e gitmemiz gerekmiyor. Dış bellek, FPGA tasarımlarında genellikle en büyük darboğazdır; bant genişliği sınırlıdır ve gecikmesi değişkendir. Model çipin içine sığdığı için bu sorun baştan ortadan kalkıyor. Bu iki gözlem, bir sonraki slayttaki güçlü yanların temeli.

35. Güçlü yanlar: minGPT'nin hesabı küçük, düzenli ve öngörülebilir; bu tam FPGA'nin sevdiği türden

Bu sayılardan sonra minGPT'nin FPGA için güçlü yanlarını altı maddede toplayabilirim. Birincisi: model kendini tekrar ediyor. Bütün layer'lar aynı hesabı farklı weight'lerle yapıyor. Bu yüzden her layer için ayrı donanım kurmak yerine tek bir veri yolu tasarlayıp layer'ları sırayla onun üzerinden geçirebiliriz; küçük bir çip için bu çok önemli. İkincisi: model çipe sığıyor. Weight'ler BRAM'de durabiliyor, dış DRAM'e gitmek gerekmiyor. Bu hem gecikmeyi düşürüyor hem de sabitliyor; çünkü dış bellek erişimi FPGA tasarımlarında genellikle hem yavaş hem de öngörülemez. Üçüncüsü: hesap tek tip. İşin yüzde 96 ila 98'i matris-vektör çarpımı. DSP bloklarından kurulmuş tek bir paralel çarpma-toplama dizisi bu işin neredeyse hepsini yapabilir. Dördüncüsü: akış tamamen öngörülebilir. Programda verinin değerine bağlı bir dallanma yok; her token, her seferinde aynı işlem sırasından geçiyor. Bu, donanımın zamanlamasını baştan, statik olarak planlayabileceğimiz anlamına geliyor; bir token'ın kaç clock cycle süreceğini hesap yapmadan bilebiliriz. Beşincisi: bazı parçalar donanımda neredeyse bedava. Embedding bir tablo okuması. Causal mask için hiçbir şey hesaplamak gerekmiyor; sadece j küçük eşit t olan konumları okumak yetiyor. GELU tek girişli bir fonksiyon olduğu için tabloyla gerçeklenebilir. Altıncısı: batch boyutu bir olduğunda, yani tek bir kullanıcı için tek tek token üretirken, GPU'lar kapasitelerinin küçük

bir kısmını kullanabiliyor; çünkü verimleri büyük batch'lerden geliyor. FPGA'de ise veri yolunu tam bu tek-token durumuna göre tasarlayabiliyoruz. Özetle: küçük bir FPGA, minGPT ölçeğindeki bir modeli dış belleğe ihtiyaç duymadan, düşük ve sabit gecikmeyle çalıştırmak için çok uygun.

36. Zayıf yanlar: non-linear işlemler, büyüyen attention ve floating point için yazılmış bir model

Madalyonun öbür yüzü, zayıf yanlar. Bunları da altı maddede topladım. Birincisi: non-linear işlemler. Matris-vektör çarpımı donanımda kolay, ama LayerNorm'da karekök ve bölme, softmax'ta üstel fonksiyon ve bölme var. Bunlar donanımda pahalı ve yavaş; her biri için özel bir hesap birimi tasarlamak gerekiyor. İşin yüzde 96'sından fazlası çarpma-toplama olsa da, tasarımın en zor kısmı bu küçük yüzde. İkincisi: floating point. Model bilgisayarda 32 bitlik floating point sayılarla eğitiliyor. FPGA'de floating point aritmetik hem çok kaynak tüketiyor hem yavaş. Doğal çözüm, sayıları daha düşük bit genişliğine indirmek; ama bunu yaparken modelin doğruluğunu korumak ayrı bir mühendislik problemi. Üçüncüsü: attention'ın maliyeti pozisyonla büyüyor. Her yeni token, saklanan bütün key'lerle karşılaştırılıyor. gpt-nano'da context 11 token olduğu için bu önemsiz, ama context yüzlerce token'a çıkınca attention darboğaz olmaya başlıyor. Dördüncüsü: KV cache belleği layer sayısı, context uzunluğu ve vektör boyutuyla çarpımsal büyüyor. minGPT her head için ayrı key ve value tutuyor; modern modellerdeki GQA bu belleği azaltıyor. Beşincisi: minGPT'nin kodu GPU için yazılmış. Bütün diziyi tek seferde, matris-matris çarpımlarıyla işliyor ve KV cache kullanmıyor. Bu yüzden kodu doğrudan donanıma çeviremiyoruz; token başına çalışan bir hesaba yeniden yazmamız gerekiyor. Altıncısı: büyümeye sınır. Model büyüdükçe weight'ler çipe sığmaz ve dış belleğe gitmek gerekir; o zaman sınırı hesap gücü değil, bellek bant genişliği belirler. Ayrıca learned position embedding, context uzunluğunu training sırasındaki block_size ile sabitliyor. Ama alttaki özet önemli: bu zorlukların hepsi önceden belli. Hangi işlemin pahalı olduğunu, nerede yeniden yazmak gerektiğini biliyoruz. Bu da minGPT'yi FPGA'de bir language model çalıştırmak için çok iyi bir başlangıç noktası yapıyor.

37. LittleBit: weight başına 0,1 bit

Son bölüm, hocamın araştırma konusu olarak verdiği makale: LittleBit. Samsung Research'ten Banseok Lee, Dongkyu Kim, Youngcheon You ve Youngmin Kim'in çalışması; NeurIPS 2025'te kabul edildi. Makalenin iddiası çok cesur: bir LLM'in weight'lerini, weight başına ortalama 0,1 bite kadar sıkıştırmak. Bunu bir an düşünelim: tek bir weight'i saklamak için en az 1 bit gerekir gibi görünüyor; 0,1 bit nasıl mümkün olabilir? Cevap, weight'leri tek tek saklamamak. Fikir üç adımda özetlenebilir. Birincisi: weight matrisini, çok daha az parametreli iki ince matrisin çarpımı olarak yazmak; buna low-rank factorization deniyor. İkincisi: bu iki ince matrisin elemanlarını yalnızca işaretlerine, yani artı bir ya da eksi bire

indirmek; her eleman bir bit. Üçüncüsü: bu indirgemede kaybolan büyüklük bilgisini, satır, sütun ve latent boyut başına birer küçük scale vektörüyle geri vermek. Bu bölümde her adımın matematiğini tek tek göstereceğim: neden SVD, scale'ler nereden geliyor, hesap nasıl yapılıyor, bit bütçesi nasıl hesaplanıyor, training nasıl yapılıyor. Sonunda da yazarların bildirdiği sonuçları ve bunun bizim FPGA projemiz için ne anlama geldiğini tartışacağım. Sonuç sayılarının hepsi makaleden; kendimiz yeniden üretmedik.

38. LittleBit'i anlamak için sekiz terim yeter; hepsi tek bir formülde buluşuyor

LittleBit'e girmeden önce bu bölümde kullanacağım sekiz terimi tek tek tanımlayayım; her kartın sağ üstünde ayrıntısının anlatıldığı slayt numarası var. Quantization: sayıları daha az seviyeyle, yani daha az bitle saklamak. Örneğin 0,25 gibi bir weight'i 8 bitlik 106 tamsayısına çevirmek. Scale: bu tamsayıları ya da işaretleri gerçek büyüklüğe geri döndüren tek bir çarpan; 106 çarpı 0,00236, yine 0,250 ediyor. Bir ses düğmesi gibi düşünebiliriz: tamsayı şekli, scale ise büyüklüğü taşıyor. Sign: bir sayının yalnızca işareti, artı bir ya da eksi bir; saklamak için tek bir bit yetiyor. Donanım için önemli bir özelliği var: artı ya da eksi bir ile çarpmak aslında toplama ya da çıkarma demek. BPW, bits per weight: bir matrisi saklamak için harcanan toplam bitin, matristeki weight sayısına bölümü; FP16'da 16, INT8'de 8, binary'de 1, LittleBit'te 0,1'e kadar. Low-rank factorization: büyük bir matrisi iki ince matrisin, U ile V transpozun çarpımı olarak yazmak. Sunumda "desen" dediğim şeyin teknik adı rank-1 component: bir sütun vektörü ile bir satır vektörünün çarpımı. SVD: bir matrisi önem sırasına dizilmiş desenlere ayıran kesin yöntem; ilk r desen, r desenle yapılabilecek en iyi yaklaşım. LoRA: fine-tuning sırasında büyük weight matrisini dondurup yalnızca değişimi low-rank olarak öğrenmek; aynı low-rank fikri ama modeli küçültmüyor, bunu ayrı bir slaytta karşılaştıracağım. Ve residual: bu bölümde residual, ilk yaklaşımın hatası demek, W eksi Ŵ_pri; ikinci bir yol bu hatayı yakalıyor. Burada bir uyarı: bu, minGPT bölümünde gördüğümüz residual connection değil; kelime aynı, anlam farklı. Alttaki formül bunların hepsini birleştiriyor: yaklaşık matrisin her elemanı, satırın scale'i h_i, sütunun scale'i g_j ve r desen üzerinden bir toplam; her desende bir scale ℓ_k ve iki işaret var. Desenler low-rank factorization'dan geliyor ve başlangıçları SVD'den; aynı yapının ikinci bir kopyası da residual yolu oluşturuyor.

39. Problem: LLM'in belleği weight'lerden gelir; 1 bitin altına inmek ancak weight'leri tek tek saklamamakla mümkün

Önce problemi tanımlayalım. Bir LLM'in belleğinin neredeyse tamamı weight'lerden geliyor. Sıkıştırmanın ölçüsü bits per weight, kısaca BPW: bir weight matrisini temsil etmek için harcadığımız toplam bit sayısının, matristeki weight sayısına oranı. Normal FP16 saklamada her weight 16 bit. Bilinen quantization yöntemleri bunu 8 bite, 4 bite, hatta her weight'i

yalnızca işaretiyle saklayarak 1 bite indiriyor. Burada kritik bir gözlem var: her weight için ayrı bir değer saklıyorsak, BPW en az 1 olmak zorunda; en az bir bitlik bilgi gerekiyor. 1'in altına inmenin tek yolu, matrisi tek tek elemanlar olarak değil bir bütün olarak, eleman sayısından çok daha az parametreyle temsil etmek. LittleBit tam olarak bunu yapıyor ve 0,1 BPW'ye kadar iniyor. Sağdaki çubuklar makalenin Tablo 3'ünden: Llama2-13B, FP16 olarak 26,06 gigabayt. 0,3 BPW'de 1,15 gigabayt, 0,1 BPW'de 0,84 gigabayt; yaklaşık 31 kat küçülme. Bir ayrıntıya dikkat: 16'dan 0,1'e inmek 160 kat demek, ama toplam küçülme 31 kat. Çünkü embedding tablosu ve lm_head sıkıştırılmıyor, FP16 kalıyor; bir önceki bölümde GPT-2'de embedding'in modelin üçte biri olduğunu görmüştük. Bunun bizim projemiz için anlamı şu: çipimizde yaklaşık 630 kilobayt BRAM var. 0,1 BPW'de bu, kabaca 50 milyon weight'e yer demek; embedding'i, activation'ları ve KV cache'i hesaba katmadan, teorik bir üst sınır olarak. Bu, bugün çalıştırdığımız modellerin yüzlerce katı.

40. Quantization sayıları az seviyeyle saklar; scale büyüklüğü geri verir; en uç hâlinde yalnızca sign kalır

LittleBit'i anlatmadan önce üç temel kavramı, quantization, scale ve sign'ı tek bir küçük örnekle göstereyim. Elimizde üç weight var: 0,12, eksi 0,30 ve 0,25. Quantization, bu sayıları daha az seviyeyle, yani daha az bitle saklamak demek. En yaygın yöntem INT8: en büyük mutlak değeri, burada 0,30'u, 127'ye eşleyen bir s sayısı seçiyoruz; s, 0,30 bölü 127, yaklaşık 0,00236. Bu s'ye scale diyoruz. Her weight'i s'ye bölüp yuvarlıyoruz: 51, eksi 127 ve 106. Bunlar 8 bitlik tamsayılar. Geri dönerken scale ile çarpıyoruz: 0,1205, eksi 0,30 ve 0,2504; göreli hata yüzde 0,15, neredeyse sıfır. Yani tamsayı şekli taşıyor, scale ise büyüklüğü; ses düğmesi benzetmesi buradan geliyor. Şekil aynı şeyi bir sayı doğrusunda gösteriyor. Üst satırda orijinal üç weight turuncu noktalar. INT8'in 255 seviyesi o kadar sık ki mavi bir bant gibi görünüyor; noktalar neredeyse hiç kaymıyor. INT4'te yalnızca 15 seviye var, çizgiler bunlar; noktalar en yakın çizgiye kayıyor ve hata yüzde 2,7'ye çıkıyor. En uç durum binary: yalnızca iki seviye kalıyor, artı alfa ve eksi alfa. Her weight'in yalnızca işaretini, sign'ını saklıyoruz; bir bit. Büyüklük için tek bir scale, alfa, kullanılıyor. Hangi alfa en iyisi? Hata karesini, w eksi alfa çarpı sign w'nin normunun karesini alfaya göre türetip sıfıra eşitlersek, alfanın mutlak değerlerin ortalaması olduğu çıkıyor; burada 0,223. Ama hata yüzde 32: üç farklı weight'in hepsi aynı büyüklüğe iniyor. Tablo bu üç durumu sayılarla veriyor. Buradan LittleBit'in neden farklı bir yol izlediği anlaşılıyor: W'yi doğrudan sign'a indirmek çok fazla bilgi kaybettiriyor. LittleBit sign'ı W'nin kendisine değil, onun low-rank factor'larına uyguluyor ve tek bir alfa yerine üç ayrı scale vektörü kullanıyor: her satır için h, her sütun için g, her desen için ℓ. Bir not: bizim FPGA tasarımımız da tam olarak buradaki INT8 ve scale mantığıyla çalışıyor.

41. LittleBit'in hilesi: matrisi sayı sayı saklamak yerine birkaç ± desen ve birkaç scale ile tarif etmek

Ayrıntılara girmeden önce LittleBit'in bütün fikrini tek slaytta, hiç formül kullanmadan söyleyeyim. Normalde bir weight matrisini sayı sayı saklarız: 4096 çarpı 4096'lık bir matris için 16,8 milyon sayı. LittleBit bunu yapmıyor; matrisi tarif ediyor. Üç hamle var. Birinci hamle: matrisi desenlere ayır. Büyük bir matris, birkaç basit desenin toplamı olarak yazılabiliyor; her desen, teknik adıyla bir rank-1 component, bir sütun vektörü ile bir satır vektörünün çarpımı. 16,8 milyon sayı yerine birkaç yüz desen. İkinci hamle: desenlerdeki sayıların yalnızca işaretini tut. Her sayının artı mı eksi mi olduğunu saklıyoruz; bu, sayı başına tek bir bit. Üçüncü hamle: işareti tutunca sayıların büyüklüğü kayboldu; onu scale'lerle geri veriyoruz. Scale, bir grup sayıyı çarpıp büyüklüğünü ayarlayan tek bir gerçek sayı; bir ses düğmesi gibi. Her satırın bir scale'i var, her sütunun bir scale'i var, her desenin bir scale'i var; makalede bunlara h, g ve ℓ deniyor. Bir mikser masası düşünün: artı-eksi desen hangi yöne itileceğini söylüyor, scale'ler ne kadar itileceğini. Sonuç olarak 4096 çarpı 4096'lık bir katman 33,6 megabayt yerine 209 kilobayta sığıyor; weight başına 0,1 bit. Üstelik hesap sırasında çarpma yerine yalnızca toplama ve çıkarma yapılıyor, çünkü artı ya da eksi bir ile çarpmak toplama ya da çıkarma demek. Bu üç hamleye iki yardımcı fikir ekleniyor: training'i rastgele değil akıllı bir noktadan başlatmak ve ilk yaklaşımın hatasını ikinci bir yolla düzeltmek. Şimdi her hamleyi önce sayısal bir örnekle, sonra matematiğiyle açacağım.

42. Hamle 1 · Bir matris, birkaç basit desenin toplamı olarak yazılabilir

Birinci hamlenin sezgisi. Üstteki 4 çarpı 4'lük matrise bakın: 16 sayı var. Ama dikkatli bakınca hepsinin sadece 8 sayıdan üretildiğini görüyoruz. Sağda yatay duran satır, [2, –1, 0,5, 1], bir desen. Matrisin her satırı bu desenin bir katı: birinci satır desenin 1 katı, ikinci satır 2 katı, üçüncü satır eksi 1 katı, dördüncü satır 3 katı. Bu katsayılar soldaki dikey sütun vektörü, [1, 2, –1, 3]. Yani matris, bir sütun vektörü ile bir satır vektörünün çarpımı: W eşittir u çarpı v transpoz; her eleman W_ij, u_i çarpı v_j. Böyle bir matrise rank 1 denir, çünkü tek bir desenden oluşuyor. 16 sayı yerine 8 sayı saklamak yeterli. Gerçek weight matrisleri tabii ki tek bir desenden oluşmuyor. Ama birkaç desenin, yani r tane desenin toplamına çok yakın olabiliyorlar: u1 v1 transpoz artı u2 v2 transpoz, böyle r tane. Bu desenleri sütun sütun yan yana koyarsak U, V matrislerini elde ediyoruz ve toplamın tamamı U çarpı V transpoz oluyor. Kazanç şu: m çarpı n sayı yerine r çarpı m artı n sayı saklıyoruz. 4096 çarpı 4096'lık bir matriste 86 desen kullanırsak, 16,8 milyon sayı yerine yaklaşık 0,7 milyon sayı; yüzde 4'ü. Bu yazımın teknik adı low-rank factorization; her desen de bir rank-1 component, rank ise matrisi kurmak için gereken bağımsız desen sayısı. Bir kazanç daha var: W çarpı x'i hesaplarken önce V transpoz çarpı x'i, sonra bunun U ile çarpımını hesaplarsak,

m çarpı n yerine r çarpı m artı n çarpma yapıyoruz. Peki en iyi desenleri nasıl buluruz? Bunun matematiksel olarak kesin bir cevabı var: SVD. Bir sonraki slayt bunun matematiği.

43. Hamle 1 · Matematik: en iyi desenleri SVD bulur; bu, low-rank factorization'dır

Birinci adım low-rank factorization. d_out çarpı d_in boyutlu bir weight matrisini, d_out çarpı r boyutlu bir U matrisi ile d_in çarpı r boyutlu bir V matrisinin transpozunun çarpımı olarak yazıyoruz; r, iki boyuttan da çok daha küçük. Şekilde mavi kare W; turuncu ince matrisler U ve V transpoz. Saklanan sayı sayısı d_out çarpı d_in yerine r çarpı d_out artı d_in oluyor. Örneğin 4096 çarpı 4096'lık bir matris 16,8 milyon sayı; r = 546 seçersek iki ince matris toplam 4,47 milyon sayı, yani yüzde 27'si. Peki U ve V'yi nasıl seçeriz ki yaklaşım en iyi olsun? Burada lineer cebirin en güzel sonuçlarından biri devreye giriyor. Her matrisin bir tekil değer ayrışımı, SVD'si var: W, σ_i u_i v_i transpoz terimlerinin toplamı; σ'lar tekil değerler, büyükten küçüğe sıralı, u_i ve v_i ortonormal vektörler. Bu toplamın yalnızca ilk r terimini alırsak W_r'yi elde ederiz. Eckart–Young teoremi şunu söylüyor: rank'ı en fazla r olan bütün matrisler arasında W'ye Frobenius normunda en yakın olanı tam olarak W_r'dir ve hata, atılan tekil değerlerin kareleri toplamının karekökü dür. Yani en iyi düşük-rank yaklaşım sorusunun kapalı bir cevabı var. Bunun pratik anlamı: bir matrisin tekil değerleri hızla küçülüyorsa, yani bilgi ilk birkaç yüz yönde yoğunlaşıyorsa, küçük bir r ile hatayı küçük tutabiliriz. LittleBit'in yazarları, LLM weight'lerindeki bu düşük-rank yapıya dayanıyor ve yüksek sıkıştırma oranlarında SVD tabanlı sıkıştırmanın pruning'den daha iyi dayandığını raporluyor. Alttaki şerit bunu bizim elle örneğimizde gösteriyor: 4 çarpı 4'lük matrisin tekil değerleri 9,66, 2,29, 0,04 ve 0. İlk desen enerjinin yüzde 94,7'sini, ikincisi yüzde 5,3'ünü taşıyor; gerisi neredeyse sıfır. Yalnız ilk deseni alırsak hata yüzde 23,1, ilk ikisini alırsak yüzde 0,4. Yani bu matris aslında iki desenden oluşuyor; hata da tam Eckart–Young'ın söylediği gibi atılan σ'ların karelerinden geliyor. Ama yalnız factorization ile 0,1 BPW'ye inemeyiz; U ve V hâlâ 16 bitlik sayılar. İkinci adım bunları bire indiriyor.

44. LoRA da low-rank kullanır ama modeli küçültmez: W'ye bir düzeltme ekler; LittleBit ise W'nin yerine geçer

Low-rank factorization deyince akla gelen en bilinen yöntem LoRA; hocamın da sorabileceğini düşündüğüm için LittleBit ile karşılaştırmak istiyorum. LoRA, Low-Rank Adaptation, Microsoft'tan Hu ve arkadaşlarının 2021 çalışması, ICLR 2022'de yayımlandı. LoRA bir sıkıştırma yöntemi değil, fine-tuning yöntemi. Hazır eğitilmiş bir modeli yeni bir göreve uyarlamak istiyoruz; bütün weight'leri yeniden eğitmek çok pahalı. LoRA şöyle yapıyor: orijinal W'yi donduruyor, hiç değiştirmiyor; yanına iki ince matrisin çarpımı olan bir düzeltme ekliyor, B çarpı A. B, d çarpı r; A, r çarpı k; r çok küçük, örneğin 8. Yalnızca A ve B eğitiliyor. B sıfırla başlatıldığı için başlangıçta model orijinaliyle birebir aynı. 4096 çarpı

4096'lık bir katmanda 16,8 milyon parametre yerine yalnızca 65 536 parametre eğitiliyor; binde 4. LittleBit ise tam tersini yapıyor: W'yi tutmuyor, kaldırıyor; yerine işaret matrisleri ve scale vektörlerinden oluşan yapıyı koyuyor. Tablo farkı özetliyor. LoRA'da low-rank olan şey weight'teki değişim; LittleBit'te matrisin kendisi. LoRA'da orijinal W aynen kalıyor; LittleBit'te kaldırılıyor. LoRA'nın factor'ları normal FP16 sayılar; LittleBit'inkiler artı-eksi bir işaretler ve scale'ler. Ve en önemlisi bellek: aynı 4096 çarpı 4096'lık katman için LoRA ile inference yaparken W'nin tamamı, 33,6 megabayt, yine bellekte; üstüne LoRA'nın 131 kilobaytı geliyor. LittleBit ile 0,1 BPW'de bütün katman 209 kilobayt. Ortak fikir şu: weight matrislerindeki bilgi az sayıda desende toplanıyor. LoRA bu fikri değişime, LittleBit matrisin kendisine uyguluyor. Bir not: QLoRA adlı bir varyant base modeli 4 bite quantize ediyor; ama o durumda bile model yaklaşık 4 BPW'de kalıyor, LittleBit'in 0,1'inden çok uzakta.

45. Hamle 2 · Desendeki her sayı, bir işaret (1 bit) ile bir scale'in çarpımı olarak yazılır

İkinci hamlenin sezgisi. Desenleri bulduk; ama desenlerin içindeki sayılar hâlâ 16 bitlik reel sayılar. LittleBit bunları da sıkıştırıyor. Fikir şu: her sayı, büyüklüğü ile işaretinin çarpımı. Bir önceki slayttaki u vektörüne bakalım: [1, 2, −1, 3]. Bunu büyüklükler [1, 2, 1, 3] ile işaretler [+, +, −, +]'nın eleman eleman çarpımı olarak yazabiliriz. İşaret tek bir bit: artı ya da eksi. Büyüklükleri ise ayrı bir vektörde, LittleBit'in dilinde bir scale vektöründe tutuyoruz: satır scale'i h. Aynısını satır desenimiz v için yapıyoruz: büyüklükler [2, 1, 0,5, 1] sütun scale'i g oluyor, işaretler [+, −, +, +]. Şimdi matrisin her elemanı dört şeyin çarpımı: satırın scale'i, sütunun scale'i ve iki işaret. Örneğin üçüncü satır ikinci sütun: 1 çarpı 1 çarpı eksi 1 çarpı eksi 1, yani 1; gerçekten de orijinal matriste o eleman 1. Burada çok önemli bir gözlem var: tek bir desen için bu yazım hiç bilgi kaybettirmiyor. Bütün büyüklük bilgisi scale'lerde, bütün yön bilgisi işaretlerde. Bunu bir mikser masası gibi düşünebiliriz: artı-eksi desen sesin hangi yöne gideceğini, scale'ler, yani mikserdeki ses düğmeleri, ne kadar açılacağını söylüyor. LittleBit'te her satırın, her sütunun ve her desenin bir scale'i var: h, g ve ℓ. Peki kayıp nerede? Kayıp, birden çok desen aynı scale'leri paylaşmak zorunda kalınca başlıyor; matematiğini gösterdikten sonra onu sayısal olarak göstereceğim.

46. Hamle 2 · Matematik: factor'lar ±1'e iner; büyüklükler satır, sütun ve latent scale'lere (h, g, ℓ) taşınır

İkinci adım binarization ve multi-scale compensation. U ve V'nin her elemanını yalnızca işaretine indiriyoruz: pozitifse artı bir, negatifse eksi bir. Böylece her eleman tek bir bit oluyor. Ama bu çok kaba bir yaklaşım; bütün büyüklük bilgisi kayboldu. LittleBit bu kaybı üç küçük scale vektörüyle telafi ediyor: h, her çıkış satırı için bir scale, uzunluğu d_out; g, her giriş sütunu için bir scale, uzunluğu d_in; ℓ ise her latent boyut için, yani r boyutun her biri için bir scale. Yaklaşık weight matrisi, ortadaki formül: diag h, U_sign, diag ℓ, V_sign

transpoz, diag g çarpımı. Köşegen matrislerle çarpmak, satırları ya da sütunları ölçeklemek demek. Şekil bunu 4 çarpı 5'lik bir matris ve rank 2 için gösteriyor: solda satır scale'leri, sonra artı-eksi işaretlerden oluşan U, iki latent scale, artı-eksi işaretlerden oluşan V transpoz ve sütun scale'leri. Değerler temsilî. Formülü eleman düzeyinde açarsak anlamı netleşiyor: Ŵ'nin i, j elemanı, h_i çarpı g_j çarpı, k üzerinden toplam ℓ_k çarpı U'nun işareti çarpı V'nin işareti. İşaretlerin çarpımı yine artı ya da eksi bir. Yani her eleman, r tane artı ya da eksi ℓ_k teriminin toplamı; sonra satırına ve sütununa göre ölçekleniyor. Burada ℓ'nin rolü SVD'deki tekil değerlere çok benziyor: hangi latent yönün ne kadar önemli olduğunu söylüyor. Saklanan bit sayısı: işaretler için r çarpı d_out artı d_in bit, her biri bir bit; scale'ler için 16 çarpı d_out artı d_in artı r bit, çünkü scale'ler FP16. r küçük ve boyutlar büyük olduğunda bu toplam, matrisin eleman sayısının çok altında kalıyor; 1'in altındaki BPW buradan geliyor. Önemli bir ayrıntı: training sırasında U ve V tam hassasiyetli olarak tutuluyor; işaretleri yalnızca forward pass'te alınıyor.

47. Kayıp, birden çok desen aynı satır scale'ini paylaşmak zorunda kalınca doğar

Tek desen için işaret artı scale yazımı kayıpsızdı. Peki bilgi nerede kayboluyor? Birden çok desen olduğunda. r desen varsa, U matrisinin her satırında r tane sayı, yani r tane büyüklük var. Ama LittleBit her satır için yalnızca bir scale, h_i, ve her desen için yalnızca bir scale, ℓ_k, saklıyor. Dolayısıyla i'nci satırın k'ncı desendeki büyüklüğünü h_i çarpı ℓ_k olarak yaklaşmak zorunda. Bu, büyüklük tablosunun rank-1 yaklaşımı: tabloyu bir satır vektörü ile bir sütun vektörünün çarpımına sığdırmak. Tablonun gerçek yapısı bu çarpıma uyuyorsa sorun yok; uymuyorsa hata doğuyor. Sağdaki tablo bunu 4 çarpı 4'lük bir örnek matriste, iki desenle gösteriyor; sayılar gerçekten hesaplandı. Soldaki iki sütun gerçek büyüklükler, sağdaki iki sütun h çarpı ℓ yaklaşımı. Birinci desen için yaklaşım iyi: 1,65'e karşı 1,67, 2,38'e karşı 2,20. Ama ikinci desenin büyüklük profili çok farklı: hemen hemen düz, 0,5 ile 0,9 arasında. Aynı satır scale'leriyle bu düz profil yakalanamıyor; birinci satırda 0,87 yerine 0,41 çıkıyor. İşte LittleBit'in kaybettiği bilgi bu. Tek desen olduğunda tablo tek bir sütun, onu h'ye birebir yazabiliyoruz; kayıp yok. LittleBit bu kaybı iki yolla azaltıyor. Birincisi training: scale'ler ve işaretler, loss'u azaltacak şekilde birlikte ayarlanıyor. İkincisi residual yol: ilk yolun hatasını, kendi ayrı scale takımı olan ikinci bir yol yakalıyor. Bir sonraki slaytlarda önce scale'lerin başlangıç değerlerinin nasıl seçildiğini, sonra bütün bunların bu küçük örnekte nasıl çalıştığını göstereceğim.

48. Dual-SVID: işaretler SVD'den, scale'ler büyüklüklerin rank-1 yaklaşımından gelir

Böyle bir modeli eğitmek için iyi bir başlangıç noktası gerekiyor; rastgele başlatırsak işaretlerle çalışan bir ağın eğitilmesi çok zor. LittleBit'in başlatma yöntemine Dual-SVID

deniyor. Dört adım var. Birincisi: weight matrisinin kesik SVD'sini alıyoruz ve tekil değerleri iki factor'a paylaştırarak W ≈ U′V′ transpoz yazıyoruz; bu, bir önceki slayttaki Eckart–Young anlamında en iyi rank-r yaklaşım. İkincisi: işaretleri bu factor'lardan alıyoruz; U_sign ve V_sign başlangıç değerleri, U′ ve V′'nin işaretleri. Üçüncüsü: geriye büyüklük matrisleri kalıyor, |U′| ve |V′|. Bunları birer rank-1 matrisle, yani bir sütun vektörü ile bir satır vektörünün dış çarpımıyla yaklaşıyoruz: |U′| yaklaşık ho çarpı ℓ_u transpoz, |V′| yaklaşık go çarpı ℓ_v transpoz. Negatif olmayan bir matrisin en iyi rank-1 yaklaşımı, örneğin en büyük tekil vektörleriyle bulunabilir. Dördüncüsü: iki latent scale'i eleman eleman çarpıp tek bir ℓo elde ediyoruz. Sağdaki kutu bunun neden tam olarak LittleBit'in yapısını verdiğini gösteriyor ve hocam bu türetmeyi özellikle göstermek istedim. Anahtar özdeşlik şu: bir A matrisini bir dış çarpımla eleman eleman çarpmak, A'yı soldan diag(h) ile, sağdan diag(ℓ) ile çarpmakla aynı şey. Eleman bazında kontrol etmek kolay: iki tarafın i, j elemanı da h_i çarpı A_ij çarpı ℓ_j. Şimdi U''yü işareti ile büyüklüğünün eleman eleman çarpımı olarak yazıyoruz. Büyüklük yerine rank-1 yaklaşımını koyunca, özdeşlik sayesinde U′ yaklaşık diag(ho) U_sign diag(ℓ_u) oluyor. Aynısı V′ için. İkisini çarpınca ortada diag(ℓ_u) çarpı diag(ℓ_v) kalıyor; köşegen matrislerin çarpımı yine köşegen ve köşegeni ℓ_u ile ℓ_v'nin eleman eleman çarpımı. Sonuç tam olarak LittleBit'in formu: diag h, U_sign, diag ℓ, V_sign transpoz, diag g. Yani Dual-SVID, LittleBit yapısının içinde, SVD çözümüne mümkün olduğunca yakın bir noktayı başlangıç olarak veriyor; quantization-aware training oradan devam ediyor. Adı da buradan geliyor: işaret ve büyüklük ayrı ayrı, iki kez ayrıştırılıyor.

49. Elle örnek: 4 × 4'lük bir matriste LittleBit ne kadar yaklaşıyor ve residual neden işe yarıyor?

Şimdi bütün parçaları küçük ve tamamen hesaplanmış bir örnekte görelim; bu slayttaki bütün sayılar gerçekten hesaplandı. Soldaki matris orijinal W, 4 çarpı 4; iki desenden ve biraz da yuvarlamadan oluşuyor. Ortadaki matris LittleBit'in tek yollu, iki desenli yaklaşımı; işaretler ve scale'ler Dual-SVID ile, yani SVD'den başlatıldı. Sağdaki matris fark. Farkın büyük olduğu hücreler, bir önceki slaytta gördüğümüz, ikinci desenin aynı satır scale'leriyle yakalanamayan kısmı. Alttaki tablo hatayı sayılarla veriyor. Referans olarak, sıkıştırmasız rank-2 SVD bu matrisi yüzde 0,4 hatayla yaklaşıyor. LittleBit'in tek yollu başlangıcı yüzde 17,7 hatada; işaretlere inmenin bedeli bu. Scale'leri ve işaretleri eğitirsek, burada doğrudan W ile Ŵ arasındaki kare hatayı küçülterek, yüzde 14,5'e iniyor; ama daha aşağı inemiyor, çünkü iki desen hâlâ aynı satır ve sütun scale'lerini paylaşıyor. Son satır çok öğretici: aynı sayıda işaret bitini iki yola bölüp her yola tek bir desen verirsek (işaret biti yine 16, scale sayısı 10'dan 18'e çıkıyor, çünkü ikinci yol kendi h ve g'sini getiriyor), hata başlangıçta bile yüzde 0,4'e iniyor, yani SVD düzeyine. Çünkü tek desenli bir yolda işaret artı scale yazımı kayıpsız; residual yol ikinci desene kendi scale takımını veriyor. Gerçek katmanlarda r yüzlerce olduğu için her yol yine birçok deseni paylaşıyor ve kayıp tamamen yok olmuyor; ama residual yolun neden işe yaradığının sezgisi bu. Dürüstlük notu: bu bizim oyuncak

hesabımız, makaledeki training loss'u değil. Ayrıca 4 çarpı 4'te bit kazancı yok, hatta scale'ler yüzünden daha fazla bit harcıyoruz; kazanç büyük matrislerde ortaya çıkıyor.

50. Hamle 3 · Ŵ hiç kurulmaz: y = Ŵx yalnızca toplama, çıkarma ve birkaç scale çarpımıyla bulunur

Üçüncü hamle hesabın kendisi ve bence FPGA için en önemli kısım. Yaklaşık matris Ŵ'yi hiç kurmamıza gerek yok. Matris çarpımının birleşme özelliği sayesinde Ŵ çarpı x'i sağdan sola, parça parça hesaplıyoruz; üstteki şerit sırayı gösteriyor. Tablo bunu elle örnekteki tek yollu Ŵ ile, gerçek sayılarla adım adım yapıyor; giriş x = [1, 2, 0, −1]. Birinci adım: x'i eleman eleman sütun scale'leriyle çarpıyoruz; dört küçük çarpma. İkinci adım, turuncu satır: V_sign transpoz ile çarpım. Bu matrisin elemanları artı ya da eksi bir olduğu için çarpma yok; yalnızca işaretlere göre topluyoruz. Birinci desenin işaretleri artı, eksi, artı, artı: 1,36 eksi 1,61 artı 0 artı eksi 0,80, sonuç eksi 1,05. İkinci desen artı, artı, artı, eksi: sonuç 3,78. Üçüncü adım: bu iki sayıyı desen scale'leriyle çarpıyoruz; iki çarpma. Dördüncü adım, yine turuncu: U_sign ile çarpım; yine yalnızca toplama ve çıkarma. Beşinci adım: satır scale'leriyle çarpıp y'yi buluyoruz. Ŵ yaklaşık olduğu için bu y, gerçek W çarpı x'ten farklı; burada önemli olan doğruluk değil, işlem türleri. Ağır kısım, yani iki büyük matris-vektör çarpımı, tamamen toplama ve çıkarmadan oluşuyor; çarpma yalnızca scale sayısı kadar. Sağdaki tablo bunu gerçek bir boyutta gösteriyor: 4096 çarpı 4096'lık bir katman 0,1 BPW'de 16,8 milyon çarpma-toplama yerine 1,41 milyon toplama-çıkarma ve yalnızca 16 556 çarpma; bellek 33,6 megabayttan 209 kilobayta. Bu sayıları makalenin formülüyle biz hesapladık. Yazarlar GPU'da da kendi 1-bit kernel'leriyle bir katmanda 11,6 kata kadar hızlanma ölçmüş; ama bu kernel düzeyinde bir ölçüm, uçtan uca değil.

51. İkinci bir yol ilk yolun hatasını yakalar; kaç desen kullanılacağı, hedef BPW'den hesaplanır

Önce bir terim uyarısı, alttaki koyu şeritte de var: buradaki residual, minGPT bölümünde gördüğümüz residual connection ile karıştırılmamalı. Orada residual, girdiyi block'un çıktısına eklemek demekti, x artı f(x). Burada ise ilk yaklaşımın hatası demek, W eksi Ŵ_pri. Kelime aynı, anlam farklı. Bu slaytta iki şey var. Birincisi residual compensation, yani ikinci düzeltme yolu. Fikir çok basit: ilk yaklaşımın bir hatası var; bu hataya aynı türden ikinci bir yaklaşım daha ekliyoruz. İkinci yolun kendi işaretleri ve kendi scale takımı var. Başlangıçta, ilk yolun hatası olan W eksi Ŵ_pri üzerine Dual-SVID uygulanarak kuruluyor; sonra iki yol birlikte eğitiliyor. Elle örnekte gördüğümüz gibi, iki desenin aynı scale'leri paylaşmasından doğan kayıp, ikinci yolun kendi scale'leri sayesinde büyük ölçüde gideriliyor. Önemli bir nokta: toplam bit bütçesi artmıyor; aynı hedef BPW iki yol arasında paylaşılıyor, dolayısıyla her yolun rank'ı daha küçük oluyor. İkincisi bit bütçesi. Bir katmanda saklanan bitleri sayalım. Her yolda işaretler r çarpı d_out artı d_in bit; scale'ler FP16, yani 16 çarpı d_out

artı d_in artı r bit. İki yol olduğu için hepsi iki ile çarpılıyor. Bunu matristeki eleman sayısına bölünce BPW formülü çıkıyor; formül r'ye göre doğrusal olduğu için ters çevirip hedef bir BPW için kaç desen kullanılabileceğini hesaplayabiliyoruz. Makalenin örneği: 4096 çarpı 4096'lık bir katman için 0,55 BPW'de 546 desen. Alttaki tablo aynı formülün farklı boyutlarda ne verdiğini gösteriyor. Elle örnekteki 4 çarpı 4'lük matris, iki desen ve tek yol ile weight başına 11 bit harcıyor; yani FP16'dan ancak biraz iyi, çünkü scale'ler baskın. 4096 çarpı 4096'da 86 desen ve iki yol ile 0,1 bit. Makaledeki 4096 çarpı 11 008'lik MLP katmanında 133 desen ile yine 0,1 bit. Ders şu: LittleBit'in kazancı matris büyüdükçe ortaya çıkıyor; bunu son slaytta bizim küçük modellerimiz için tekrar ele alacağım.

52. Training: sign'ın türevi sıfırdır; LittleBit geri yayılımda onu yumuşak bir rampa gibi davranarak öğrenir

Böyle bir modeli eğitmenin önünde ciddi bir matematiksel engel var: sign fonksiyonu. Grafikte kesikli mavi çizgi sign: eksi tarafta eksi bir, artı tarafta artı bir, ortada bir basamak. Bu fonksiyonun türevi her yerde sıfır; basamağın olduğu yerde ise tanımsız. Training gradient ile yapılıyor; türev sıfırsa gradient U ve V'ye hiç ulaşamaz ve işaretler hiç değişmez. Model öğrenemez. Klasik çözüm straight-through estimator, kısaca STE: forward'da gerçek sign'ı kullan, backward'da türevini sanki bir mış gibi davran. LittleBit biraz daha incelikli bir yol kullanıyor: SmoothSign. Forward'da yine gerçek sign; backward'da ise tanh(100x)'in türevi. Grafikteki koyu turuncu eğri tanh(100x): sign'a o kadar yakın ki neredeyse basamak, ama sıfır civarında yumuşak ve türevlenebilir. Karşılaştırma için açık renkle tanh(5x) de çizili. Türevi 100 çarpı 1 eksi tanh kare; sıfırın hemen yanındaki değerler için büyük, uzaktakiler için sıfıra yakın. Yani işaretini değiştirmeye yakın olan sayılar güçlü bir sinyal alıyor. Yazarlar bunun ultra düşük bitte STE'den daha iyi çalıştığını raporluyor. Training'in ikinci bileşeni knowledge distillation: elimizde orijinal FP16 model var, öğretmen o. LittleBit modeli öğrenci. Loss iki terimden oluşuyor: öğrencinin çıkış dağılımı ile öğretmeninki arasındaki KL divergence ve ara katman çıktıları arasındaki ortalama kare hata, 10 ağırlıkla. Yani öğrenci, öğretmeni hem sonuçta hem yol boyunca taklit ediyor. Training boyunca U ve V gerçek sayı olarak tutuluyor; forward'da yalnızca işaretleri kullanılıyor. Eğitim verisi WikiText-2 ve C4'ün bazı parçaları; 5 epoch, dizi uzunluğu 2048.

53. Sonuçlar: model 0,1 BPW'de bile çalışıyor, ama 0,3'ten 0,1'e geçişte kalite belirgin düşüyor

Yazarların bildirdiği sonuçlara bakalım; bu sayıları kendimiz yeniden üretmedik, hepsi makalenin tablolarından. Soldaki tablo WikiText-2 üzerinde perplexity; düşük olan daha iyi. FP16 Llama2-7B'nin perplexity'si 5,47. LittleBit 1 BPW'de 9,08, 0,55'te 10,47, 0,3'te 12,00 ve 0,1'de 15,92. Karşılaştırma için STBLLM, 1 bitin altındaki önceki en iyi yöntem: 0,55 BPW'de 30,67, 0,3 BPW'de 1 800 civarı, yani model fiilen çökmüş. LittleBit'in asıl katkısı burada:

aynı bit bütçesinde, 1 bitin altında çok daha kullanılabilir bir model. Sağ tarafta bellek: Llama2-13B, 26 gigabayttan 0,84 gigabayta, yani 31 kat küçülüyor; 70B modelde küçülme 70 kata yaklaşıyor, çünkü büyük modelde sıkıştırılmayan embedding'in payı daha küçük. Hız tarafında yazarlar kendi 1-bit kernel'leriyle bir katmanda 11,6 kata kadar hızlanma ölçmüş; ama bu kernel düzeyinde, uçtan uca bir inference hızı değil. Dürüstçe söylemem gereken sınırlar da var. Birincisi, kalite kaybı küçük değil: 0,1 BPW'de perplexity FP16'nın yaklaşık üç katı; 0,3 ile 0,1 arasında belirgin bir uçurum var. İkincisi, tam 1 bitte LittleBit en iyisi değil; BinaryMoS 7,74 ile daha iyi. Üçüncüsü, residual yol her zaman yardımcı değil: küçük bir model olan OPT-1.3B'de 0,1 BPW'de perplexity'yi 11,5 artırıyor. Yazarlar büyük modellerde yararlı olduğunu söyleyip bütün deneylerde kullanıyor. Bu son gözlem bizim için önemli; çünkü bizim modellerimiz çok daha küçük.

54. FPGA için: çarpıcısız matris-vektör çarpımı çok uygun, ama küçük katmanlarda scale'lerin bit maliyeti baskın

Son olarak LittleBit'in bizim projemiz için ne anlama geldiği. Önce fırsatlar. Birincisi: LittleBit'in ağır işi, elemanları artı-eksi bir olan matrislerle matris-vektör çarpımı ve bu, çarpma değil toplama ya da çıkarma. FPGA'de çarpma için DSP blokları kullanıyoruz ve bunlar sınırlı; toplama-çıkarma ise LUT'larla kurulabiliyor ve LUT'lardan on binlerce var. İkincisi: işaretler bir bit; BRAM'e 8 bitlik weight'lere göre sekiz kat yoğun paketlenebiliyor. Üçüncüsü: hesap akışı yine matris-vektör çarpımı, yani KV cache ile tek token işlediğimiz generation akışıyla tamamen uyumlu. Dördüncüsü: bellek; daha önce hesapladığımız gibi, 630 kilobaytlık BRAM 0,1 BPW'de teorik olarak 50 milyon weight'e yer açıyor. Ama burada formülden çıkan ve bence önemli bir zorluk var. LittleBit'te her yolun h ve g vektörleri FP16 ve sayıları d_out artı d_in. İki yol için yalnızca bu scale'lerin BPW'ye katkısı 32 çarpı d_out artı d_in bölü d_out çarpı d_in. Büyük bir LLM katmanında, 4096 çarpı 4096'da, bu 0,016 BPW; ihmal edilebilir. 512 çarpı 512'de 0,125; 0,1 hedefini tek başına aşıyor. gpt-nano'nun 48 çarpı 48'lik katmanlarında ise 1,33 BPW: daha işaretleri saklamadan, yalnızca scale'ler 1 biti geçiyor. Yani LittleBit büyük matrisler için tasarlanmış; küçük modellerde aynı sıkıştırma oranları matematiksel olarak mümkün değil. Bu, araştırma sorularına götürüyor. Bir: küçük modellerde LittleBit'in anlamlı BPW alt sınırı ne ve bu sınırda kalite nasıl? Makaledeki en küçük model OPT-1.3B ve orada bile residual yol 0,1 BPW'de zarar veriyordu. İki: binary bir matris-vektör birimi FPGA'de kaç LUT tüketir ve token başına kaç cycle sürer; DSP'ye dayalı bir tasarıma göre ne kazandırır? Üç: makalede scale'ler FP16; FPGA'de floating point pahalı olduğu için, bu scale'leri integer'a indirip yazılımla bit bit aynı sonucu veren bir donanım kurulabilir mi? Bu üç soru, LittleBit'i bu projeye bağlamanın doğal yolu.

55. son

Toparlamak için dört cümle. Birincisi: minGPT yeni bir mimari değil, GPT-2'nin sade ve okunabilir hâli; embedding, her biri attention ve MLP'den oluşan L tane aynı block ve bir lm_head, hepsi residual connection ile bağlı. İkincisi: embedding, matematikte bir one-hot vektörle matris çarpımı ama pratikte bir tablo okuması; training'de gradient yalnızca dizide geçen token'ların satırlarına gidiyor ve eğitilen tablolar görevin yapısını geometrik olarak kodluyor. Attention ise geçmişteki token'ların, ağırlıkları içeriğe göre hesaplanan bir ortalaması; causal mask ve KV cache sayesinde generation her adımda tek bir token işliyor. Üçüncüsü: bu yapı bir FPGA için çok uygun; zorluklar da belli: karekök, üstel fonksiyon ve bölme gibi non-linear işlemler, floating point ve uzun context'te büyüyen attention. Dördüncüsü: araştırma konusu olarak incelediğimiz LittleBit, weight matrislerini low-rank factor'lara ayırıp işaretlerine indirerek ve küçük scale vektörleriyle telafi ederek 0,1 BPW'ye iniyor. Ağır işi çarpıcı gerektirmeyen bir matris-vektör çarpımı; bu da FPGA için çok doğal bir araştırma hedefi, küçük modellerde scale maliyeti gibi açık sorularla birlikte. Teşekkür ederim; sorularınızı bekliyorum.

Ek · Olası sorular ve kısa cevaplar

Skorları neden $\sqrt{d_h}$'ye bölüyoruz? q ve k'nin bileşenleri bağımsız, ortalaması 0 ve varyansı 1 ise dot product d_h tane çarpımın toplamıdır; varyansı d_h olur. Head boyutu büyüdükçe score'lar büyür, softmax tek bir token'a kilitlenir ve gradient'ler söner. $\sqrt{d_h}$'ye bölmek varyansı 1'e sabitler. gpt-nano'da $d_h = 16$, yani çarpan 1/4.

Pre-LN nedir, neden kullanılıyor? LayerNorm, attention ve MLP'ye girmeden önce uygulanıyor; residual yolun kendisi normalize edilmiyor. Böylece residual stream üzerinden gradient doğrudan akabiliyor ve derin modellerde training daha kararlı oluyor. GPT-2 ve minGPT bu düzeni kullanıyor.

MLP neden dört kat genişliyor? Orijinal Transformer makalesinden gelen deneysel bir tercih. Sonucu şu: her block'taki parametrelerin yaklaşık üçte ikisi MLP'de. gpt-nano'da bir block 28 272 parametre; bunun 18 672'si MLP.

Attention'ın maliyeti neden "karesel" deniyor? T token'lık bir dizide her token kendinden önceki bütün token'larla karşılaştırılıyor; toplam yaklaşık $T^2/2$ dot product. KV cache ile generation sırasında token başına maliyet doğrusal: t'inci token t+1 key ile karşılaştırılıyor.

KV cache ne kadar bellek ister? Her layer'da her pozisyon için bir key ve bir value vektörü: toplam $2 \cdot L \cdot T \cdot C$ sayı. gpt-nano'da $2 \times 3 \times 11 \times 48 = 3\,168$ sayı; çok küçük. Uzun context'li ve çok layer'lı modellerde ise belleğin önemli bir kısmını kaplıyor.

Greedy decoding'de softmax neden gerekmiyor? Softmax monoton artan bir fonksiyon; logit'lerin sırasını değiştirmiyor. En büyük logit her zaman en büyük olasılığa karşılık geliyor. Bu yüzden greedy decoding'de doğrudan en büyük logit seçilebilir.

minGPT ile GPT-2 arasındaki fark yalnızca ölçek mi? Mimari aynı; minGPT, OpenAI'nin GPT-2 weight'lerini from_pretrained ile yükleyip çalıştırabiliyor. Farklar parametre sayısı, training verisi ve kodun hız için optimize edilmemiş olması (KV cache yok, FlashAttention yok).

Model sıralamayı gerçekten "anlıyor" mu? Model sıralama algoritması bilmiyor; altı rakamlık dizilerde bir sonraki token'ın dağılımını öğreniyor. 729 girdinin hepsinde doğru sonuç veriyor ve attention weight'lerinde girdideki ilgili rakamlara baktığını görebiliyoruz. Ama başka uzunluklarda test edilmedi; context de block_size ile sınırlı.

Neden FPGA, neden GPU değil? GPU'lar büyük batch'lerde çok yüksek throughput sağlar. Tek kullanıcı ve tek token'lık generation'da kapasitelerinin küçük bir kısmını kullanırlar. FPGA'de veri yolu tam bu duruma göre tasarlanabiliyor: düşük ve sabit gecikme, düşük güç ve gömülü sistemlere uygunluk. Bu, minGPT ölçeğindeki küçük modeller için anlamlı bir hedef.

Embedding'in bir Linear katmandan farkı ne? Matematikte farkı yok: embedding, girişi one-hot olan ve bias'ı olmayan bir Linear katman. Fark girişin yapısında: one-hot vektörde tek bir 1 olduğu için çarpım tek bir satır seçimine iniyor, gradient de yalnızca o satıra gidiyor (sparse gradient). Bu yüzden PyTorch onu çarpım olarak değil, indeksleme olarak hesaplıyor.

Token ve position embedding neden toplanıyor, yan yana eklenmiyor (concat)? Toplama vektör boyutunu C'de tutuyor; concat boyutu büyütürdü. Yüksek boyutlu bir uzayda iki bilgi farklı yönlere yerleşebiliyor ve sonraki Linear katmanlar onları ayırabiliyor. Eğittiğimiz gpt-nano'da token ve position vektörleri arasındaki ortalama |cosine| 0,20; yani büyük ölçüde farklı yönlerde duruyorlar.

LittleBit'te weight başına 0,1 bit nasıl mümkün? Weight'ler tek tek saklanmıyor. Matris, $r(d_{out} + d_{in})$ tane işaret biti ve birkaç küçük FP16 scale vektörüyle temsil ediliyor. BPW, bu toplam bitin matristeki eleman sayısına bölümü; r küçük ve matris büyük olduğunda 1'in çok altına iniyor.

LittleBit ile LoRA arasındaki fark ne? İkisi de low-rank fikrine dayanıyor ama amaçları ters. LoRA bir fine-tuning yöntemi: W'yi donduruyor, yanına low-rank bir düzeltme B·A ekliyor ve yalnızca onu eğitiyor; inference'ta W tam boyutunda yine bellekte (4096 × 4096'da 33,6 MB + 131 KB). LittleBit bir sıkıştırma yöntemi: W'yi kaldırıyor, yerine ±1 işaret matrisleri ve scale vektörleri koyuyor (aynı katman 0,1 BPW'de 209 KB).

Buradaki residual, transformer'daki residual connection ile aynı şey mi? Hayır. Transformer'da residual connection, girdiyi block'un çıktısına eklemek: x + f(x). LittleBit'te residual, ilk yaklaşımın hatası: W – Ŵ_pri. Bu hatayı aynı yapıda ikinci bir yol yaklaşıyor. Kelime aynı, anlam farklı.

Neden W doğrudan sign'a indirilmiyor? Tek bir scale ile binary quantization çok bilgi kaybettiriyor: slayt 40'taki üç weight'te göreli hata %32, çünkü bütün weight'ler aynı büyüklüğe iniyor. LittleBit sign'ı W'nin low-rank factor'larına uyguluyor ve satır, sütun ve desen başına ayrı scale'ler kullanıyor.

LittleBit neden SVD ile başlıyor? Eckart–Young teoremine göre rank'ı en fazla r olan matrisler arasında W'ye Frobenius normunda en yakın olanı, SVD'nin ilk r terimi. Dual-SVID bu en iyi yaklaşımın işaretlerini alıyor, büyüklüklerini de rank-1 yaklaşımla h, g ve ℓ scale'lerine çeviriyor.

sign fonksiyonunun türevi sıfırken model nasıl eğitiliyor? Forward'da gerçek sign kullanılıyor; backward'da onun yerine tanh(100x)'in türevi, yani $100(1 - \tanh^2(100x))$ kullanılıyor (SmoothSign). Training sırasında U ve V tam hassasiyetli tutuluyor, işaretleri yalnızca forward'da alınıyor.

LittleBit bizim küçük modellerimize uygulanabilir mi? Aynı oranlarla değil. İki yolun h ve g scale'leri tek başına $32(d_{out} + d_{in})/(d_{out} \cdot d_{in})$ BPW tutuyor: 4096 × 4096'da 0,016, ama gpt-nano'nun 48 × 48 katmanlarında 1,33. Küçük modellerde anlamlı alt sınırın ne olduğu, açık bir araştırma sorusu.

LittleBit'te embedding neden sıkıştırılmıyor? Yazarlar embedding ve lm_head katmanlarını FP16 bırakıyor; yalnızca transformer block'ları sıkıştırılıyor. Büyük vocabulary'li modellerde bu tablolar büyük olduğu için toplam küçülme 160 kat değil, Llama2-13B'de 31 kat.

Training'i de FPGA'de yapabilir miyiz? Prensipte evet, ama backpropagation, optimizer durumu ve yüksek hassasiyetli aritmetik gerekir; donanım çok daha karmaşık olur. Bu projenin kapsamı inference: model bilgisayarda eğitiliyor, FPGA'de çalıştırılıyor.